

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Dr. Indu Singh CSE DTU

CS- 411

CYBER VULNERABILITY AND ETHICAL HACKING UNIT - 2



Dr. Indu Singh
Assistant Professor (DTU - CSE)

Syllabus

SYLLABUS (UNIT - 2)

System Hacking: Understanding the Password Hacking Techniques, Rootkits, Trojans, Backdoors, Viruses and Worms, Sniffers, Denial of Service, Session Hijacking

Foundational Concepts

Before diving into specific attack types, a shared vocabulary: what counts as an attack, an exploit, and the difference between network and computer security.

Core Definitions

Attack

Any unauthorized attempt to access, damage, or use network resources or computer systems, typically by exploiting a weakness.

Exploit

A specially crafted piece of data or code designed to take advantage of a specific vulnerability.

Network Security

Securing devices that are part of a networked infrastructure — anything connected to other systems.

Computer Security

Securing a stand-alone device that is not connected to a network.

Why This Field Keeps Growing

- The FBI, CIA, and Interpol all flag computer crime as the fastest-growing crime type worldwide.
- It is far easier to attack a company from a living room than to physically break into their building.
- Even when investigators trace a crime to a specific machine, shared access to that machine can make it impossible to pin the act on one individual with certainty.
- Until deterrents catch up with the ease of attacking remotely, demand for security professionals stays high.
- Case in point: FBI agents once raided a home expecting to seize a computer holding drug-transaction records — the occupant destroyed the hard drive with a shotgun the moment they arrived, and investigators still had enough evidence elsewhere to proceed.

Password Hacking Techniques

Every technique an attacker uses to discover, steal, or bypass a victim's login credentials. Passwords remain the most common — and most heavily targeted — authentication factor in use today.

How It Works

1. Information Gathering (Reconnaissance)

The attacker identifies targets and gathers information to increase the efficiency of the attack. They look for patterns, job titles, social media profiles, or previous data breaches.

- **Aim:** Obtain clues about a user's password habits (e.g., favourite sports team, pet names, birth year).
- **Example:** An attacker finds a corporate employee's LinkedIn profile and notices they frequently post about being a die-hard "Mumbai Indians" cricket fan.

2. Password Harvesting

The attacker collects potential passwords based on gathered intelligence or by using lists of previously leaked credentials from other websites.

- **Goal:** Build a "wordlist" or "dictionary" of likely passwords.
- **Example:** Using the research from Phase 1, the attacker creates a custom list containing variations like MumbaiIndians2026, MI_Cricket_Fan, or HitmanRohit1.

How It Works

3. Attack Execution

The attacker uses software tools to automate the process of testing the passwords against the target system.

- **Common Techniques:**
 - **Brute Force:** Testing every possible character combination (slow, but guaranteed if the password is short).
 - **Dictionary Attack:** Testing thousands of common words and phrases.
 - **Credential Stuffing:** Using stolen usernames/passwords from one breach to log into other unrelated sites, relying on the fact that people reuse passwords.
- **Example:** The attacker uses a tool like *Hydra* or *Hashcat* to run the custom list against the employee's company email portal.

How It Works

4. Privilege Escalation

Once the attacker successfully guesses one password, the attack isn't always over. They attempt to gain more access within the network.

- **Goal:** Move from a low-level user account to an administrative account with higher permissions.
- **Example:** After compromising the employee's email, the attacker finds a saved document titled "Server Passwords" on the internal cloud drive, allowing them to gain control over the entire company server.

5. Cleanup and Persistence

To avoid detection, the attacker removes evidence of the breach and establishes a "backdoor" to ensure they can return even if the original password is changed.

- **Goal:** Maintain long-term access while remaining invisible to security software (SIEM).
- **Example:** The attacker deletes the temporary login logs from the system and creates a new, hidden "service account" with administrative rights, ensuring they don't need to guess the original user's password ever again.

Types of Password Hacking

Dictionary Attack

Runs a pre-built list of common or leaked words against a login or stolen hash until one matches.

Brute-Force Attack

Systematically tries every possible character combination until the correct password turns up.

Hybrid Attack

Mixes dictionary words with numbers/symbols (e.g. "Summer2024!") to catch users who "customize" a common word.

Rainbow Table Attack

Uses a precomputed hash-to-password table to reverse a stolen hash almost instantly.

Credential Stuffing

Reuses username/password pairs leaked from one breach against many other sites, betting on reuse.

Phishing

Tricks the victim into typing their real password into a fake but convincing login page.

Types of Password Hacking (cont.)

Keylogging

Records every keystroke the victim types — hardware or software — to capture the password as entered.

Shoulder Surfing

Simply watches the victim type their password, in person or via camera.

Social Engineering

Manipulates a person directly (e.g. posing as IT support) into revealing their password.

Default-Credential Exploitation

Tries factory-default logins (admin/admin) that were never changed after setup.

Real-World Example

REAL-WORLD EXAMPLE

In the 2012 LinkedIn breach, millions of password hashes were stolen. Because they were hashed with unsalted SHA-1, attackers cracked a very large share of them quickly using dictionary and brute-force methods, then reused the recovered passwords on other sites via credential stuffing.

Prevention Measures

- Enforce strong, lengthy password policies and block common/leaked passwords.
- Add multi-factor authentication (MFA) so a stolen password alone isn't enough.
- Hash passwords with a slow, salted algorithm (bcrypt/scrypt/Argon2), never fast unsalted hashes.
- Lock accounts or throttle attempts after repeated failed logins.
- Train users against phishing and shoulder-surfing.
- Use a password manager to avoid reuse across sites.
- Change every default credential before a device goes into production.

Types of Password Hacking

1. Dictionary Attack

A **Dictionary Attack** is a technique used by attackers to break into password-protected systems by systematically trying a pre-compiled list of common words, phrases, and previously compromised passwords.

How It Works

Stage	Action	Real-World Numerical Example (LinkedIn Breach)
1. Preparation	The attacker obtains or creates a "dictionary" file—a text document containing thousands of common passwords (e.g., "password123", "admin", "qwerty").	The attacker downloads a leaked password dictionary containing 14 million commonly used passwords (e.g., 123456, password, linkedin, welcome, password123).
2. Selection	The attacker selects a target system or a database of encrypted passwords (hashes).	From the leaked LinkedIn database, the attacker selects one user whose SHA-1 password hash is <code>cbfdac6008f9cab4083784cbd1874f76618d2a97</code> .
3. Iteration	An automated script takes each word from the dictionary, encrypts it (or hashes it), and compares it to the target password/hash.	The attack tool hashes each dictionary word and compares it with the target hash. Attempt 1: 123456 → No match Attempt 25: password → No match Attempt 432: welcome → No match Attempt 2,184: password123 → Generated hash matches the leaked hash
4. Matching	If a match is found, the attack stops, and the attacker gains access using that password.	The attacker discovers that the user's password is password123 after 2,184 attempts . Using the recovered password, the attacker can log into the victim's LinkedIn account and potentially other accounts if the same password was reused.

Prevention Measures

- **Implement Strong Password Policies:** Enforce the use of long, complex passwords that include a mix of uppercase/lowercase letters, numbers, and special symbols, making them impossible to include in a standard dictionary list.
- **Account Lockout Mechanisms:** Configure systems to temporarily lock an account after a set number of failed login attempts, rendering automated dictionary scripts ineffective.
- **Multi-Factor Authentication (MFA):** This is the most effective defense; even if an attacker successfully guesses the password via a dictionary attack, they still cannot gain access without the second factor (like a mobile app code or security key).
- **Rate Limiting:** Slow down the login process for each failed attempt. If a system introduces a delay (or "throttle") after each wrong guess, a dictionary attack that might take seconds could end up taking years to complete.
- **Use Salting:** When storing passwords, systems should "salt" the hashes (adding random data to each password before hashing). This ensures that even if an attacker gets the password database, they cannot use pre-computed dictionaries (like "rainbow tables") to reverse the hashes instantly.

Types of Password Hacking

2. Brute Force Attack

A **Brute Force Attack** is a cryptographic attack where an attacker systematically submits every possible combination of characters until the correct password or decryption key is found. Unlike a dictionary attack, which uses a list of common passwords, brute force is an exhaustive, "guess-everything" approach.

How It Works

Stage	Action	Example	Real-World Numerical Example
1. Target Identification	The attacker identifies a target system, service, or encrypted file to crack.	Identifying an encrypted backup file or a remote login portal.	The attacker targets an online banking login page with the username john.doe@example.com .
2. Exhaustive Generation	The attacker uses software to generate every possible character combination (e.g., a, b, c... aa, ab, ac...).	A script generating passwords starting from 1 character up to 10+ characters.	The attacker assumes the password is 6 lowercase letters . There are $26^6 = 308,915,776$ possible combinations (e.g., aaaaaa, aaaaaab, aaaaac, ..., zzzzzz).
3. Continuous Testing	Each generated combination is tested against the target password/key.	Automatically attempting thousands of login requests per minute.	The attack tool submits 500 login attempts per second (assuming no rate limiting). If the correct password is "secure", it may be found after testing 180,000,000 combinations.
4. Success/Failure	The process runs until the correct match is found or the attacker gives up.	The script finally finds the matching key, granting the attacker access.	At 500 attempts/second , testing 180,000,000 passwords takes $180,000,000 \div 500 = 360,000$ seconds $\approx$ 100 hours (≈ 4.2 days) . If account lockout occurs after 5 failed attempts, the attack fails.

Prevention Measures

- **Complexity Requirements:** Enforcing long, complex passwords significantly increases the time required for a brute force attack to succeed, often making it computationally infeasible.
- **Account Lockout Policies:** Configuring systems to lock accounts after a specific number of failed login attempts effectively stops automated brute force tools.
- **Multi-Factor Authentication (MFA):** Even if a brute force attack successfully guesses a password, MFA provides an essential second layer of security that the attacker cannot bypass with just a password.
- **Rate Limiting:** Introducing delays or limiting the number of login attempts allowed from a specific IP address prevents attackers from testing combinations at high speed.
- **Strong Encryption Standards:** Using robust, modern encryption algorithms makes the underlying "keys" much harder to break, even with massive computing power.

Types of Password Hacking

3. Hybrid Attack

A **Hybrid Attack** combines elements of both **Dictionary Attacks** and **Brute Force Attacks** to increase the chances of successfully cracking a password. It is generally more efficient than a pure brute force attack and more effective than a dictionary attack alone.

How It Works

Stage	Action	Description
1. Seed List	Start with a dictionary of common words.	The attacker uses a "seed" file containing popular words (e.g., "admin", "password", "sun").
2. Mutation	Apply rules to modify the seed words.	The attacker's software automatically appends, prepends, or substitutes characters (e.g., changing "sun" to "sun123", "S-u-n!", or "P@ssword").
3. Testing	Systematically test the mutated list.	Each modified word is tested against the target system as a potential password.
4. Expansion	Shift to full Brute Force if needed.	If the mutated dictionary fails, the attacker falls back to a traditional brute force approach to guess remaining combinations.

Example

Stage	Action	Real-World Numerical Example
1. Seed List	Start with a dictionary of common words.	The attacker loads a dictionary containing 1 million common passwords such as <code>password</code> , <code>admin</code> , <code>welcome</code> , <code>sun</code> , and <code>football</code> .
2. Mutation	Apply rules to modify the seed words.	For each word, the attacker applies 50 mutation rules (e.g., <code>password</code> → <code>Password</code> , <code>password1</code> , <code>Password123</code> , <code>P@ssword</code> , <code>password@123</code>). This creates $1,000,000 \times 50 = 50,000,000$ password candidates.
3. Testing	Systematically test the mutated list.	The attacker tests the 50 million generated passwords against a leaked password database. The user's password is Password123 , which is found after 12,500,000 attempts.
4. Expansion	Shift to full Brute Force if needed.	If none of the 50 million mutated passwords match, the attacker switches to brute force, trying all 8-character combinations (e.g., $26^8 \approx 208.8$ billion lowercase combinations).

Why it is Effective

Many users create passwords by taking a common word and adding a few digits or symbols at the end (e.g., Summer2026!).

- A **pure dictionary attack** would miss this because "Summer2026!" might not be in the pre-compiled list.
- A **pure brute force attack** would eventually find it, but it would take a very long time to guess all the character combinations.
- A **hybrid attack** finds it quickly by taking the word "Summer" from the dictionary and applying the rule to add "2026!".

Prevention Measures

- **Implement Strong Password Policies:** Enforce longer, more complex passwords that do not rely on simple words or predictable patterns, making them resistant to mutation rules.
- **Multi-Factor Authentication (MFA):** This remains the strongest defense, as it prevents the attacker from gaining access even if they successfully crack the password.
- **Account Lockout & Rate Limiting:** These measures prevent the attacker from testing thousands of mutated password variations in a short amount of time.
- **Use Salting:** When storing passwords, using a unique "salt" for each user ensures that an attacker cannot use pre-computed tables or quickly crack multiple passwords if they obtain the database.

Types of Password Hacking

4. Rainbow Table Attack

A **Rainbow Table Attack** is a highly efficient cryptographic attack that uses pre-computed tables to reverse cryptographic hash functions, typically used to recover plain-text passwords from a database of stolen password hashes.

How It Works

Stage	Action	Description
1. Pre-computation	Creating the table	Attackers generate massive tables containing millions of hashes and their corresponding plain-text passwords in advance.
2. Hash Retrieval	Obtaining the target	The attacker gains access to a database of encrypted password hashes (e.g., from a data breach).
3. Lookup	Finding the match	Instead of calculating hashes in real-time like brute force, the attacker simply performs a rapid lookup in their pre-computed table.
4. Recovery	Revealing the password	Once a match is found, the plain-text password associated with the hash is revealed instantly.

Mathematical Workflow

Assume:

Pre-computed password hashes in the rainbow table =
500,000,000

Password hashes obtained from the breached database =
2,000,000

Passwords successfully recovered = **300,000**

Stage 1: Pre-computation

Password hashes stored in the rainbow table:
 $T = 500,000,000$

Stage 2: Hash Retrieval

Password hashes extracted from the compromised database:

$$H = 2,000,000$$

Hash retrieval rate:

$$\frac{2,000,000}{500,000,000} \times 100 = 0.4\%$$

Stage 3: Lookup

Hashes successfully matched using the rainbow table:

$$L = 300,000$$

Lookup success rate:

$$\frac{300,000}{2,000,000} \times 100 = 15\%$$

Stage 4: Recovery

Plain-text passwords successfully recovered:
 $R = 300,000$

Recovery rate:

$$\frac{300,000}{300,000} \times 100 = 100\%$$

If **25%** of the recovered passwords were reused across other online services:

$$C = 0.25 \times 300,000$$

$$C = 75,000 \text{ accounts}$$

Result: 75,000 additional online accounts could potentially be compromised through password reuse after the attackers recovered 300,000 plain-text passwords from the stolen password hashes.

Prevention Measures

- **Password Salting:** This is the primary defense. By adding a unique, random string of data (the "salt") to each password before it is hashed, the attacker's pre-computed tables become useless, as the salts make the resulting hashes unique to each user.
- **Use Strong Hashing Algorithms:** Utilizing modern, computationally expensive hashing algorithms (like Argon2 or bcrypt) makes the pre-computation process extremely difficult and time-consuming for attackers.
- **Multi-Factor Authentication (MFA):** As with other password-based attacks, MFA ensures that even if an attacker successfully recovers a password using a rainbow table, they still cannot access the account without the secondary factor.

Types of Password Hacking

5. Credential Stuffing

Credential Stuffing is a type of cyberattack where an attacker takes a list of usernames and passwords stolen from one data breach and automatically "stuffs" them into the login pages of other websites and services.

How It Works

Stage	Action
1. Data Acquisition	The attacker buys or obtains a database of leaked credentials (usernames and passwords) from a previous data breach of a different site.
2. Automation	Using specialized software (bots), the attacker automatically tests these stolen credential pairs against a new target website.
3. Exploitation	The attack relies on the fact that many users engage in "password reuse"—using the same password across multiple accounts.
4. Unauthorized Access	When a pair works, the attacker gains access to the user's account on the target site.

Example Scenario: Nintendo Credential Stuffing Attack (2020)

Target Details

Item	Value
Target	Nintendo Accounts
Attack Method	Credential Stuffing
Credentials Source	Previous data breaches
Accounts Compromised	~300,000
Goal	Unauthorized account access

Stage	Action	Real-Life Example (Nintendo 2020)
1. Data Acquisition	Obtain leaked credentials	Attackers collected millions of username/password pairs from previous data breaches available online.
2. Automation	Test credentials automatically	Bots attempted millions of login requests on Nintendo accounts using the stolen credentials.
3. Exploitation	Reused passwords succeed	Users who reused the same password on multiple websites unknowingly allowed attackers to log in successfully.
4. Unauthorized Access	Gain account control	Approximately 300,000 Nintendo accounts were accessed, exposing personal information and payment details.

Mathematical Workflow

Assume:

- Leaked credentials = **10,000,000**
- Credentials tested = **2,000,000**
- Successful logins = **300,000**

Stage 1: Data Acquisition

Leaked credentials obtained:

$$L = 10,000,000$$

Stage 2: Automation

Credentials tested:

$$T = 2,000,000$$

Testing rate:

$$\frac{2,000,000}{10,000,000} \times 100 = 20\%$$

Stage 3: Exploitation

Successful logins:

$$S = 300,000$$

Success rate:

$$\frac{300,000}{2,000,000} \times 100 = 15\%$$

Stage 4: Unauthorized Access

If **20%** of compromised accounts had payment information stored:

$$P = 0.20 \times 300,000$$

$$P = 60,000 \text{ accounts}$$

Prevention Measures

- **Implement Multi-Factor Authentication (MFA):** This is the most effective defense; even if an attacker has the correct username and password, they still cannot log in without the second authentication factor.
- **Enforce Unique Passwords:** Use a password manager to ensure you use a different, complex password for every single account you own, which neutralizes the risk of a single breach affecting all your accounts.
- **Bot Detection Tools:** Website owners should deploy advanced bot detection and management solutions that identify and block automated login attempts.
- **Monitor for Breaches:** Regularly check if your email or passwords have appeared in known data breaches (using sites like "Have I Been Pwned") and change them immediately if they have.
- **Credential Screening:** Companies can use services that check if a user's chosen password has already been compromised in known breaches before allowing them to use it.

Types of Password Hacking

6. Phishing

Phishing is a deceptive cyberattack where attackers pose as a trusted entity to trick individuals into revealing sensitive information, such as login credentials, credit card numbers, or personal data.

How It Works

Stage	Action	Description
1. Targeting	The attacker identifies a target or a large group of potential victims.	Attackers may use public information or data from leaks to craft believable lures.
2. Lure Creation	Designing the deceptive message.	Emails, SMS, or messages are crafted to look like they come from legitimate sources, often creating a sense of urgency.
3. Distribution	Sending the message.	Messages are sent via email, text (smishing), or phone (vishing) to the victims.
4. Exploitation	The victim interacts with the lure.	The victim clicks a malicious link, downloads a harmful attachment, or enters information into a fake website.
5. Data Theft	Capturing the sensitive data.	The stolen information is transmitted directly to the attacker for exploitation.

Example Scenario: Google & Facebook Phishing Scam

Victim Details

Item	Value
Target Companies	Google & Facebook
Attacker	Fake Supplier (posing as Quanta Computer)
Attack Method	Phishing Emails
Goal	Fraudulent Money Transfer
Financial Loss	Over \$100 million

Stage	Action	Real-Life Example (Google & Facebook Scam)
1. Targeting	Identify victims	The attacker targeted employees responsible for processing vendor invoices at Google and Facebook.
2. Lure Creation	Create a convincing message	Fake invoices and emails were created using the name and branding of Quanta Computer , a legitimate supplier.
3. Distribution	Send phishing emails	The fraudulent invoices were emailed to finance employees, appearing to be genuine business requests.
4. Exploitation	Victim trusts the email	Employees approved and processed the fake invoices, believing they were legitimate vendor payments.
5. Data Theft / Fraud	Achieve the objective	More than \$100 million was transferred to bank accounts controlled by the attacker before the fraud was detected.

Mathematical Workflow

Assume:

Employees targeted = 500

Phishing emails sent = 500

Employees opening email = 200

Fake invoices approved = 25

Average invoice value = \$4 million

Stage 1: Targeting

Employees identified:

$$N = 500$$

Stage 2: Lure Creation

Emails prepared:

$$L = 500$$

Stage 3: Distribution

Emails opened:

$$O = 200$$

Open rate:

$$\frac{200}{500} \times 100 = 40\%$$

Stage 4: Exploitation

Approved invoices:

$$A = 25$$

Approval rate:

$$\frac{25}{200} \times 100 = 12.5\%$$

Stage 5: Financial Fraud

Average payment:

$$\$4,000,000$$

Total financial loss:

$$\text{Loss} = 25 \times \$4,000,000$$

$$\boxed{\text{Loss} = \$100,000,000}$$

Prevention Measures

- **Verify the Source:** Always check the sender's email address or phone number for subtle discrepancies that indicate it is not from a legitimate domain.
- **Avoid Clicking Links:** Instead of clicking links in suspicious messages, navigate directly to the official website by typing the address into your browser.
- **Enable Multi-Factor Authentication (MFA):** MFA adds an essential layer of security that prevents attackers from accessing your account even if they successfully steal your password.
- **Be Skeptical of Urgency:** Phishing attacks often use high-pressure tactics, such as threats of account suspension or claims of unauthorized activity, to force quick, unthinking reactions.
- **Use Security Tools:** Modern email services often have built-in spam and phishing filters that automatically identify and block known malicious messages.

Types of Password Hacking

7. Keyloggers

A keylogger is a tool or software that secretly records keystrokes on a device, capturing sensitive information such as passwords, messages, and financial data.

Hardware vs. Software Keyloggers

Software Keyloggers

Behave essentially like a virus or Trojan — installed on the OS, logging keystrokes silently in the background.

Hardware Keyloggers

Tiny devices (often under an inch) installable in under 30 seconds, plugged in-line between the keyboard cable and the USB port.

Wi-Fi-Enabled Variants

Many hardware keyloggers now include built-in Wi-Fi, letting an attacker pull stored keystrokes remotely without ever touching the device again.

Named Products

KeyGrabber and KeyGhost are real commercial examples; KeyGrabber reveals a hidden flash drive with LOG.TXT via a special key combo.

How it Works

1. Delivery & Infection

The attacker must first get the keylogger onto your system.

- **Method:** This is usually achieved through **phishing emails** with malicious attachments, **drive-by downloads** from compromised websites, or by bundling the keylogger with "free" or "cracked" software.
- **Example:** You receive an email that looks like a legitimate invoice. When you open the attached PDF, a script runs in the background that silently installs the keylogger software on your computer.

2. Installation & Persistence

Once inside, the keylogger hides itself to ensure it survives reboots and remains undetected.

- **Method:** It may hook into the Operating System's API (the interface between your keyboard and your computer) or hide deep within the system kernel, making it invisible to basic task managers.
- **Example:** The malware registers itself as a "system service," so it automatically launches every time you turn on your computer, mimicking a standard background driver.

How it Works

3. Monitoring & Interception

This is the "active" phase where the keylogger records your activity.

- **Method:** It listens to every keystroke you make and often correlates it with the application you are using. Advanced versions can also take screenshots of your desktop or record clipboard data.
- **Example:** You open your banking website. As you type your username and password, the keylogger records those exact sequences in a hidden file on your hard drive, along with the URL of the site you are visiting.

4. Data Exfiltration

The attacker needs to get the collected data off your computer.

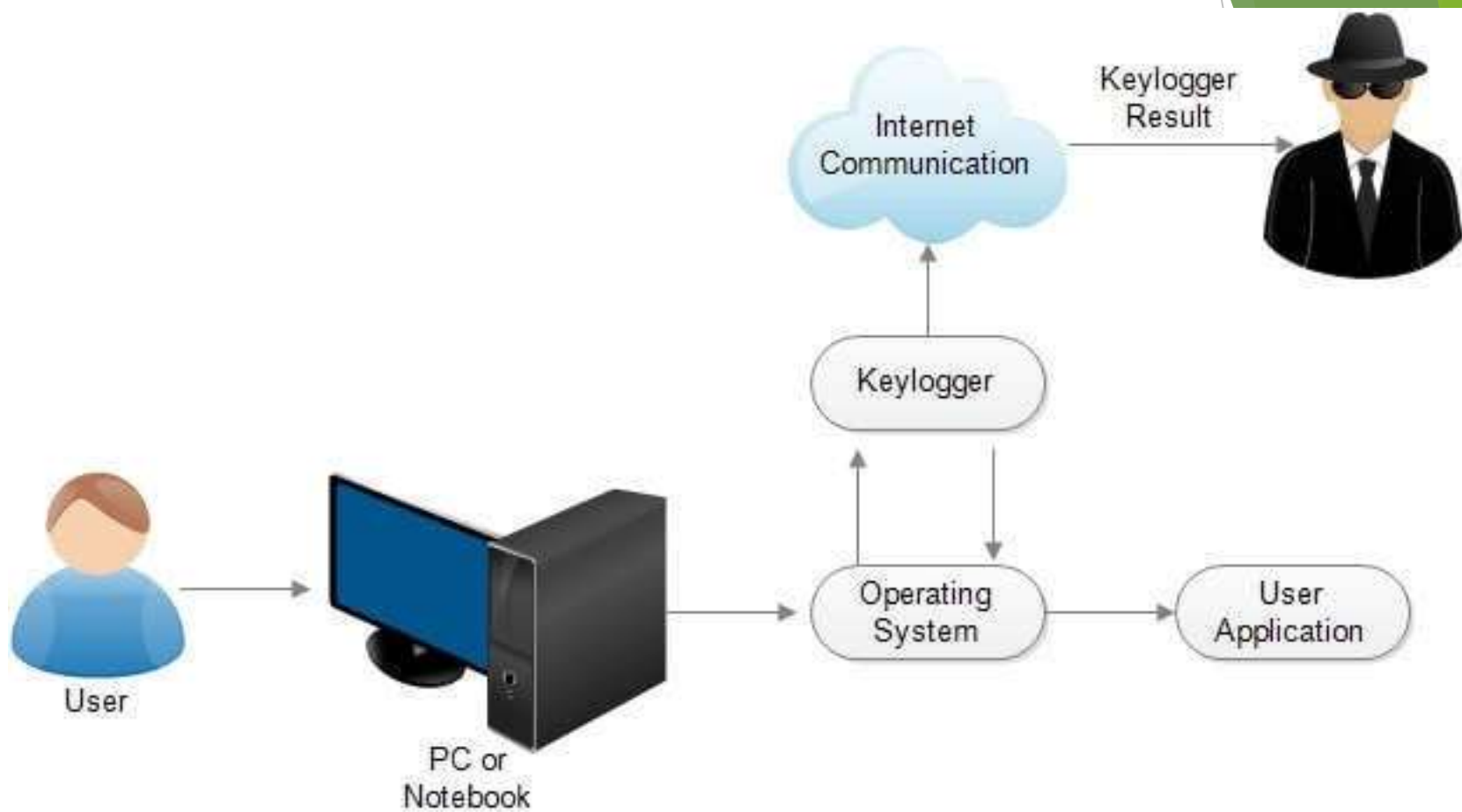
- **Method:** The keylogger periodically sends the stored logs to the attacker's **Command-and-Control (C&C) server** via the internet.
- **Example:** Every night at 2:00 AM, the keylogger initiates a small, encrypted network connection to a remote server, uploading the "log file" containing your typed credentials.

How it Works

5. Exploitation

The attacker now has your raw data and uses it to fulfill their goal.

- **Method:** Because they have your actual passwords, they do not need to "break" anything; they simply log in as you.
- **Example:** The attacker uses the stolen banking credentials to log into your account from their own device, bypasses security questions (since they know the answers because you typed them), and initiates a fraudulent wire transfer.



Example Scenario: Zeus Keylogger

Victim Details

Item	Value
Victim PC	Windows 10
Victim IP	192.168.1.25
Banking Website	https://securebank.com
Attacker C2 Server	203.0.113.50
Malware	Zeus (Zbot)

Stage	Action	Real-Life Example (Zeus Keylogger)
1. Delivery & Infection	Malware reaches the victim.	Alice opens a phishing email attachment (Invoice.pdf.exe), which silently installs the Zeus keylogger.
2. Installation & Persistence	Keylogger hides and starts automatically.	Zeus installs itself as a background service and automatically starts whenever Windows boots.
3. Monitoring & Interception	Records user activity.	Alice visits https://securebank.com and types her username and password. Zeus records every keystroke along with the website URL.
4. Data Exfiltration	Sends collected data to the attacker.	At scheduled intervals, Zeus securely sends the recorded keystrokes to the attacker's C2 server (203.0.113.50) .
5. Exploitation	Uses stolen credentials.	The attacker logs into Alice's online banking account using the captured credentials and performs unauthorized transactions.

Mathematical Workflow

Assume:

- Emails sent = 100
- Victims opening attachment = 20
- Successful installations = 10
- Credentials captured = 8
- Successful account compromises = 6

Stage 1: Delivery & Infection

Emails delivered:

$$E = 100$$

Victims opening attachment:

$$O = 20$$

Open rate:

$$\frac{20}{100} \times 100 = 20\%$$

Stage 2: Installation & Persistence

Successful installations:

$$I = 10$$

Installation success:

$$\frac{10}{20} \times 100 = 50\%$$

Stage 4: Data Exfiltration

Credentials successfully uploaded:

$$U = 8$$

Upload success:

$$\frac{8}{8} \times 100 = 100\%$$

Stage 4: Data Exfiltration

Credentials successfully uploaded:

$$U = 8$$

Upload success:

$$\frac{8}{8} \times 100 = 100\%$$

Stage 5: Exploitation

Successful account compromises:

$$C = 6$$

Success rate:

$$\frac{6}{8} \times 100 = 75\%$$

If each compromised account loses

₹50,000:

$$\text{Total Loss} = 6 \times ₹50,000 = ₹300,000$$

100 Phishing Emails Sent



20 Users Open Attachment



10 PCs Infected



8 Banking Credentials Captured



Credentials Sent to C2 Server



6 Bank Accounts Compromised



Total Loss = ₹300,000

Legitimate Use vs. Malicious Use

- Legitimate use: during an authorized security test, keyloggers can capture passwords — but only with written client permission first.
- Also used legitimately by organizations, parents, and law enforcement/forensics for monitoring.
- Malicious use: a disgruntled employee could plant one on a manager's computer to harvest confidential information later.
- Physical hardware versions require repeat physical access to retrieve the log — locked offices make this harder.
- A Wi-Fi-enabled keylogger removes that repeat-access requirement entirely.
- Defense tip: during routine physical checks, look for unfamiliar hardware plugged in along the keyboard cable.

Types of Password Hacking

8. Shoulder Surfing

Shoulder surfing is a form of social engineering where an attacker observes a victim to gain unauthorized information, such as passwords, PINs, or sensitive data, by literally looking over the victim's shoulder.

How It Works

Stage	Action	Description
1. Positioning	The attacker gains a vantage point.	They position themselves near a victim in a public space to get a clear view of a screen or keyboard.
2. Observation	The attacker watches the victim's input.	They watch as the victim enters credentials on a laptop, smartphone, or at an ATM.
3. Recording	The attacker captures the information.	The attacker either memorizes the information, takes a covert photo/video, or uses a nearby hidden camera.
4. Exploitation	The attacker uses the stolen data.	The stolen credentials or PINs are used to access the victim's accounts or perform fraudulent transactions.

Example Scenario: ATM Shoulder Surfing Victim Details

Item	Value
Victim	Alice
ATM Location	Shopping Mall
ATM PIN	4582
ATM Card	Debit Card
Attacker	Standing Behind Victim

Stage	Action	Real-Life Example (ATM Shoulder Surfing)
1. Positioning	Gain a vantage point	The attacker stands directly behind Alice in the ATM queue to get a clear view of the keypad.
2. Observation	Watch the victim's input	As Alice enters her PIN (4582), the attacker carefully observes the keypad and finger movements.
3. Recording	Capture the information	The attacker memorizes the PIN or secretly records it using a smartphone or hidden camera.
4. Exploitation	Use the stolen data	After obtaining Alice's ATM card (e.g., through theft or card skimming), the attacker uses the observed PIN (4582) to withdraw money from her account.

Mathematical Workflow

Assume:

Customers observed at ATMs = 5,000, PINs successfully recorded= 1,000, Successful ATM card thefts = 150

Stage 1: Positioning

Customers targeted:

$$C = 5,000$$

Stage 2: Observation

PINs observed:

$$O = 1,000$$

Observation rate:

$$\frac{1,000}{5,000} \times 100 = 20\%$$

Stage 3: Recording

Successfully recorded PINs:

$$R = 900$$

Recording success rate:

$$\frac{900}{1,000} \times 100 = 90\%$$

Stage 4: Exploitation

Successful fraudulent withdrawals:

$$E = 150$$

Exploitation success rate:

$$\frac{150}{900} \times 100 = 16.7\%$$

If the average stolen amount was £800 per compromised account:

$$L = 150 \times 800$$

$$L = £120,000$$

Total financial loss = £120,000

Prevention Measures

- **Shield Your Input:** When entering sensitive data in public, use your body or hand to shield the keyboard or screen from view.
- **Be Aware of Your Surroundings:** Stay alert to people standing too close behind you or loitering near terminals like ATMs or point-of-sale systems.
- **Use Biometrics:** Where possible, use biometric authentication like fingerprint or facial recognition, which does not require typing sensitive passwords in public.
- **Privacy Screens:** Use privacy filters on your laptop or mobile device screens, which make it difficult for anyone not directly in front of the screen to read what is displayed.
- **Avoid Sensitive Tasks in Public:** Whenever possible, avoid accessing banking apps or entering login credentials while in crowded, high-traffic areas.

Types of Password Hacking

9. Social Engineering

Social Engineering is the art of manipulating people into performing actions or divulging confidential information that can be used for malicious purposes. Rather than hacking technical systems directly, social engineers target the "human element" of security, exploiting psychological triggers like trust, fear, urgency, or helpfulness.

How It Works

Stage	Action	Description
1. Information Gathering	Reconnaissance	The attacker researches the target using public information (social media, corporate websites) to build a credible pretext.
2. Pretexting/Building Trust	Establishing the "Hook"	The attacker contacts the victim, posing as a trusted figure (like IT support, a colleague, or a vendor) to gain their confidence.
3. Exploitation	Triggering the Action	Using psychological pressure (e.g., "urgently verify your password or lose access"), the attacker convinces the victim to divulge data or perform a harmful action.
4. Execution	Achieving the Goal	The attacker uses the obtained information to access systems, move money, or install malware.

Example Scenario: Twitter Social Engineering Attack (2020) Target Details

Item	Value
Company	Twitter
Victim	Twitter Employee
Attacker	Fake IT Support
Goal	Access Internal Admin Panel
Outcome	High-profile accounts compromised

Stage	Action	Real-Life Example (Twitter Hack 2020)
1. Information Gathering	Reconnaissance	Attackers collected employee information from LinkedIn, social media, and public websites.
2. Pretexting / Building Trust	Establishing the Hook	Attackers called employees pretending to be Twitter IT Support and requested login credentials.
3. Exploitation	Triggering the Action	Employees entered their credentials into a fake login portal, allowing attackers to steal them.
4. Execution	Achieving the Goal	Attackers accessed Twitter's admin panel and posted Bitcoin scam tweets from verified accounts like Apple, Elon Musk, and Barack Obama.

Mathematical Workflow

Assume:

Employees identified through reconnaissance = **5,000**

Employees contacted via social engineering = **500**

Employees who disclosed credentials = **50**

Stage 1: Information Gathering

Employees identified:

$$I = 5,000$$

Stage 2: Pretexting / Building Trust

Employees contacted:

$$P = 500$$

Contact rate:

$$\frac{500}{5,000} \times 100 = 10\%$$

Stage 3: Exploitation

Employees who entered credentials into the fake login portal:

$$E = 50$$

Credential theft success rate:

$$\frac{50}{500} \times 100 = 10\%$$

Stage 4: Execution

Compromised employee accounts used to access internal systems:

$$C = 30$$

Execution success rate:

$$\frac{30}{50} \times 100 = 60\%$$

If **80%** of the compromised accounts had administrative privileges that enabled attackers to post scam tweets:

$$A = 0.80 \times 30$$

$$A = 24 \text{ administrative accounts}$$

Prevention Measures

- **Security Awareness Training:** Regular education helps individuals recognize the common tactics and psychological triggers used by social engineers.
- **Verify Identity:** Never provide sensitive information or grant access based on an unsolicited request; always verify the requester's identity through an official, independent channel.
- **Enable Multi-Factor Authentication (MFA):** MFA provides a critical safety net that prevents unauthorized access even if an attacker successfully tricks a user into giving up their password.
- **Skepticism of Urgency:** Be wary of any communication that pressures you to act immediately, especially if it involves sensitive data or financial transactions.
- **Principle of Least Privilege:** Organizations should limit access to sensitive systems so that a successful social engineering attack on one employee has a limited impact.

Types of Password Hacking

10. Default-Credential Exploitation

Default-Credential Exploitation is a vulnerability where an attacker gains unauthorized access to a system by using the factory-set usernames and passwords that came with the hardware or software.

How It Works

Stage	Action	Description
1. Discovery	Identifying targets	Attackers use automated tools to scan for devices or software that still use well-known, factory-set credentials.
2. Targeting	Analyzing vulnerabilities	The attacker identifies systems where default logins (e.g., admin/admin) have not been changed.
3. Access	Attempting authentication	The attacker logs in using the documented default credentials.
4. Exploitation	Gaining control	With administrative access, the attacker can reconfigure the device, steal data, or use the device to breach a larger network.

Example Scenario: Mirai Botnet Using Default Credentials Target Device

Item	Value
IoT Camera IP	192.168.1.100
Service Port	23 (Telnet)
Default Username	admin
Default Password	admin
Attacker Server	203.0.113.50

Stage	Action	Real-Life Example (Mirai Botnet)
1. Discovery	Identifying targets	Mirai scans the Internet and finds an IP (192.168.1.100) with Telnet (Port 23) open.
2. Targeting	Analyzing vulnerabilities	The bot identifies that the IoT camera still uses the default credentials admin/admin .
3. Access	Attempting authentication	Mirai logs in successfully using admin/admin without exploiting any software vulnerability.
4. Exploitation	Gaining control	The malware installs itself, connects to the Command & Control (C2) server (203.0.113.50), and the camera becomes part of the Mirai botnet for DDoS attacks.

Mathematical Workflow

Assume:

IoT devices scanned = **1,000,000**, Devices with default Telnet credentials = **50,000**, Devices successfully infected = **40,000**

Stage 1: Discovery

IoT devices scanned:

$$D = 1,000,000$$

Stage 2: Targeting

Devices found with Telnet enabled and default credentials:

$$T = 50,000$$

Target identification rate:

$$\frac{50,000}{1,000,000} \times 100 = 5\%$$

Stage 3: Access

Devices successfully accessed using the default **admin/admin** credentials:

$$A = 40,000$$

Authentication success rate:

$$\frac{40,000}{50,000} \times 100 = 80\%$$

Stage 4: Exploitation

Compromised devices added to the Mirai botnet:

$$B = 40,000$$

Botnet enrollment rate:

$$\frac{40,000}{40,000} \times 100 = 100\%$$

If **75%** of the infected devices participated in a DDoS attack:

$$D_a = 0.75 \times 40,000$$

$$D_a = 30,000 \text{ devices}$$

Prevention Measures

- **Change Default Passwords:** The most critical step is to immediately change the default username and password to a strong, unique value upon installation.
- **Disable Unnecessary Services:** Turn off remote management features or services that are not required for normal operation.
- **Update Firmware/Software:** Regularly update devices to the latest firmware, as manufacturers may include security patches that enforce credential changes.
- **Network Isolation:** Keep sensitive devices on a separate network or behind a firewall to restrict unauthorized access attempts.
- **Multi-Factor Authentication (MFA):** Whenever possible, enable MFA on administrative interfaces to add a layer of security that default credentials cannot bypass.

Rootkits

Stealth software installed after an attacker has already gained access, built to hide the attacker's continued presence from the user and from normal security tools.

Types of Rootkits

User-mode / Application Rootkit

Runs at normal application privilege, replacing standard utilities; easiest to detect.

Kernel-mode Rootkit

Intercepts core OS system calls, far harder to detect or remove.

Bootkit (Boot-loader Rootkit)

Infects the master boot record, loading before the OS itself and surviving reinstalls.

Firmware/Hardware Rootkit

Hides in BIOS/UEFI or device firmware, surviving even a hard-drive swap in some cases.

Hypervisor Rootkit

Runs beneath the OS as a virtual-machine layer, so the "real" OS never realizes it's being controlled.

Library Rootkit

Patches shared system libraries so every program that calls them is silently manipulated.

How It Works

1. Initial Compromise

A rootkit usually requires an existing security vulnerability to gain a foothold.

- **Method:** The attacker exploits a known software flaw (unpatched OS or application) or uses a Trojan (malware disguised as legitimate software) to gain initial access.
- **Example:** A user downloads a "cracked" version of a premium software suite. The installer contains a malicious payload that gains enough permission to initiate the next, deeper installation phase.

2. Installation & Privilege Escalation

The goal here is to elevate permissions to the highest level (Kernel mode) and install the malicious files in areas where standard security software rarely looks.

- **Method:** The malware modifies system boot files, device drivers, or the kernel itself. Because it installs at a level equal to or *below* the operating system, it can "lie" to the OS.
- **Example:** The rootkit replaces a legitimate system driver with a malicious version. Now, whenever the OS asks the driver for a list of running processes or files, the driver filters out the rootkit's own processes before showing the results to the user.

How It Works

3. Hiding

This is what makes rootkits unique; they prioritize hiding their own existence.

- **Method:** They intercept calls made by the OS to the hardware or file system (Hooking). If you try to open the folder where the rootkit lives, the rootkit tells the OS the folder is empty.
- **Example:** You open your Task Manager to check for suspicious activity. The rootkit intercepts the request and ensures that its own malicious processes are deleted from the list before the Task Manager displays it on your screen.

4. Establishing Persistence & Backdoors

Once hidden, the rootkit ensures it survives system restarts and allows the attacker a permanent way back in.

- **Method:** It creates a hidden backdoor (a remote access portal) that bypasses standard login screens or firewall rules.
- **Example:** The rootkit opens a secret, encrypted communication channel on a non-standard port. The attacker can now connect to your computer from anywhere in the world, treating your machine like their own personal server.

How It Works

5. Payload Execution

With full control and invisibility, the attacker uses the machine for their specific goals.

- **Method:** Since they have "system-level" access, they can do anything: steal data, install more malware (like ransomware), or use your computer as part of a "botnet" to attack other websites.
- **Example:** The attacker uses your machine to launch a massive Distributed Denial of Service (DDoS) attack against a third-party website, using your bandwidth and computing power while you remain completely unaware.

Example Scenario: Sony BMG XCP Rootkit Victim System

Item	Value
Victim PC	Windows XP Computer
Victim IP	192.168.1.25
Software	Sony Music CD (XCP Copy Protection)
Hidden Driver	\$sys\$drv.sys
Attacker/Software Vendor	Sony BMG XCP

Stage	Real-World Example (Sony BMG XCP Rootkit)
1. Initial Compromise	User inserts a Sony music CD, which silently installs the XCP rootkit on Windows.
2. Installation & Privilege Escalation	XCP installs a kernel-level driver (\$sys\$drv.sys) with SYSTEM privileges.
3. Hiding	Files and processes beginning with "\$sys\$" are hidden from Windows Explorer and Task Manager.
4. Persistence & Backdoors	The driver loads automatically at every boot, remaining active and difficult to remove.
5. Payload Execution	Attackers exploit the hiding feature to conceal malware, steal data, install ransomware, or use the PC in a botnet.

Mathematical Workflow

Assume:

Sony XCP CDs inserted into Windows PCs = 100,000

Systems where the rootkit was successfully installed = 80,000

Systems remaining infected after reboot = 72,000

Stage 1: Initial Compromise

Target systems exposed by inserting the Sony XCP CD:

$$C = 100,000$$

Stage 2: Installation & Privilege Escalation

Systems where the kernel-level driver (\$sys\$drv.sys) was successfully installed with SYSTEM privileges:

$$I = 80,000$$

Installation success rate:

$$\frac{80,000}{100,000} \times 100 = 80\%$$

Stage 3: Hiding

Systems where the rootkit successfully hid files and processes beginning with "\$sys\$":

$$H = 76,000$$

Hiding success rate:

$$\frac{76,000}{80,000} \times 100 = 95\%$$

Stage 4: Persistence & Backdoors

Systems where the malicious driver automatically loaded at every system boot:

$$P = 72,000$$

Persistence rate:

$$\frac{72,000}{76,000} \times 100 = 94.7\%$$

Stage 5: Payload Execution

Compromised systems exploited for malicious activities:

$$E = 18,000$$

Payload execution rate:

$$\frac{18,000}{72,000} \times 100 = 25\%$$

If 40% of these exploited systems were used to steal sensitive user information:

$$S = 0.40 \times 18,000$$

$$S = 7,200 \text{ systems}$$

Real-World Example

REAL-WORLD EXAMPLE

The 2005 Sony BMG rootkit scandal — Sony shipped music CDs whose copy-protection software secretly installed rootkit-style concealment on customers' Windows PCs. Researchers later showed the same hiding technique could be, and was, exploited by unrelated malware to hide on the same infected machines.

Prevention Measures

- Use dedicated anti-rootkit/boot-time scanners rather than relying only on regular antivirus.
- Enable Secure Boot/UEFI integrity checks to block unauthorized boot-loader changes.
- Use file-integrity monitoring to compare critical system files against a known-good baseline.
- Patch the OS and firmware promptly to close the initial-access holes rootkits rely on.
- Tightly restrict and monitor privilege escalation, since installing a rootkit requires admin/root access.
- Treat a confirmed rootkit infection as unrecoverable by partial cleanup — wipe and reinstall from trusted media.

Trojans

Malware disguised as legitimate or desirable software to trick the victim into installing and running it voluntarily. Trojans don't self-replicate — they rely entirely on deception.

Types of Trojans

Remote Access Trojan (RAT)

Gives the attacker full remote control of the victim's machine (e.g. Back Orifice).

Backdoor Trojan

Opens a hidden channel letting the attacker bypass normal login entirely.

Banking / Infostealer Trojan

Targets online-banking sessions, saved credentials, and browser-stored data.

Downloader Trojan

Does little harm itself but quietly fetches and installs further malware.

DDoS Trojan

Turns the machine into a "zombie" that joins a botnet used to flood a target on command.

Ransom Trojan

Encrypts files and demands payment, delivering ransomware via a Trojan disguise.

Types of Trojans (cont.)

Fake-AV Trojan

Poses as antivirus software, showing fake infection alerts to scare the victim into paying or installing more malware.

How It Works

1. Delivery & Initial Disguise

- **Action:** Tricking the user into downloading a malicious file disguised as legitimate software or a document.
- **Example:** Sending an email with an attachment named `Payroll_Update.pdf.exe`, relying on the user to mistake the executable for a standard PDF.

2. User Execution & Installation

- **Action:** The user triggers the hidden payload, allowing the malware to copy files and create startup tasks.
- **Example:** The user double-clicks the file, triggering a script that secretly installs a malicious service in `System32` and edits the registry to run on startup.

3. Persistence & Communication

- **Action:** The Trojan hides itself and establishes a connection to a remote server.
- **Example:** The malware injects its code into a legitimate process like `explorer.exe` and opens a hidden network tunnel to the attacker's Command & Control server.

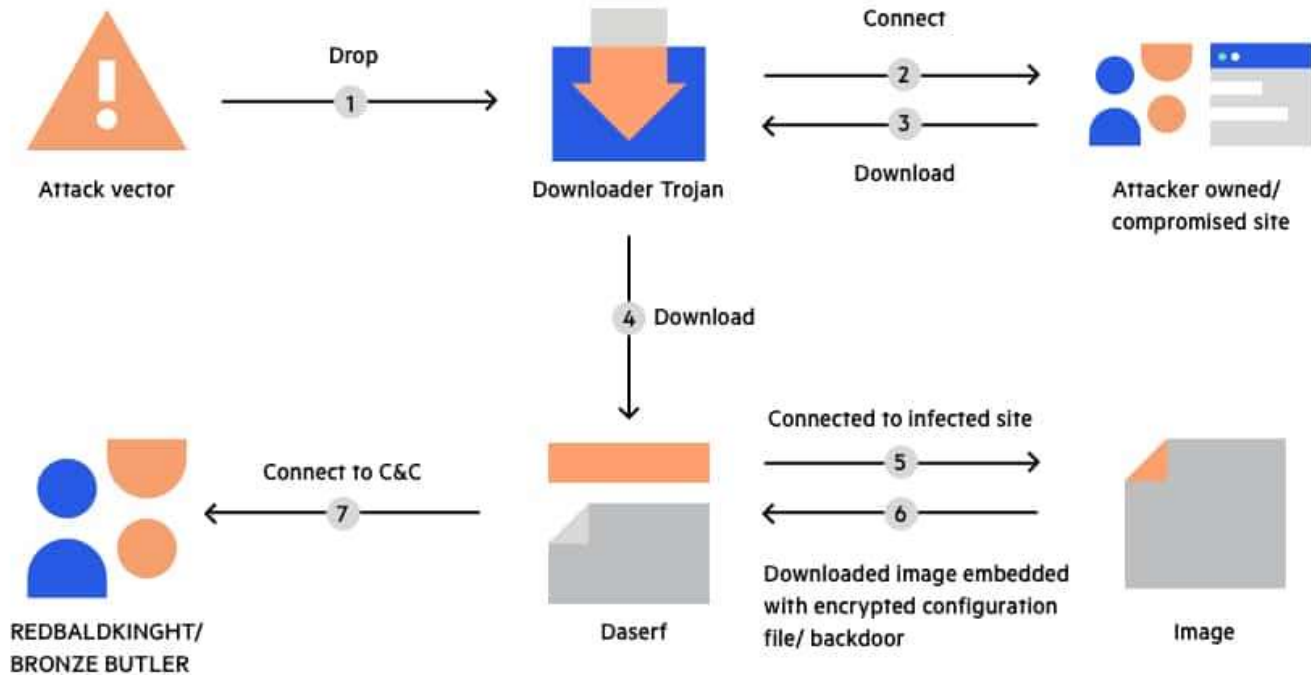
How It Works

4. Payload Execution

- **Action:** The malware performs its primary task, such as stealing data or providing backdoor access.
- **Example:** The Trojan monitors your activity and sends a screenshot of your screen, capturing your login information for a banking website.

5. Covering Tracks & Exploitation

- **Action:** Hiding the malicious activity from the user and security software while the attacker profits.
- **Example:** The Trojan modifies your local firewall rules to allow its traffic and hooks the Windows API to hide its process from the Task Manager list.



Example Scenario: Zeus Banking Trojan

Victim System

Item	Value
Victim PC	Windows 10 Computer
Victim IP	192.168.1.25
Attacker C2 Server	203.0.113.50
Fake Attachment	Payroll_Update.pdf.exe
Target Website	https://securebank.com

Stage	Action	Real-Life Example (Zeus Trojan)
1. Delivery & Initial Disguise	The Trojan is disguised as a legitimate file.	Alice receives an email titled " Monthly Payroll Update " with the attachment Payroll_Update.pdf.exe . Because Windows hides known file extensions, it appears to be a normal PDF document. Alice downloads the file to 192.168.1.25 .
2. User Execution & Installation	The user executes the Trojan, allowing it to install.	Alice double-clicks the attachment. Instead of opening a PDF, the Zeus Trojan installs itself in the background, copies its files to the system, and configures itself to start automatically whenever Windows boots.
3. Persistence & Communication	The Trojan remains active and contacts its command server.	After installation, Zeus injects its code into a legitimate Windows process (such as explorer.exe) and establishes an encrypted connection to the attacker's Command & Control (C2) server at 203.0.113.50 to receive instructions.
4. Payload Execution	The Trojan performs its primary malicious activity.	Alice visits https://securebank.com and logs in. Zeus captures the banking credentials entered into the browser and sends the stolen information to the attacker's C2 server.
5. Covering Tracks & Exploitation	The Trojan hides its activity while the attacker uses the stolen data.	Zeus continues running silently in the background, avoiding user detection while periodically communicating with the C2 server. The attacker later uses the stolen banking credentials to perform unauthorized transactions or other fraudulent activity.

Mathematical Workflow

Assume:

Phishing emails delivered = **500,000**

Users who executed the Trojan = **25,000**

Banking credentials successfully stolen = **5,000**

Stage 1: Delivery & Initial Disguise

Phishing emails successfully delivered:

$$D = 500,000$$

Stage 2: User Execution & Installation

Users who opened the malicious attachment and installed the Zeus Trojan:

$$I = 25,000$$

Installation rate:

$$\frac{25,000}{500,000} \times 100 = 5\%$$

Stage 3: Persistence & Communication

Infected systems that successfully connected to the Command & Control (C2) server:

$$C = 22,500$$

Communication success rate:

$$\frac{22,500}{25,000} \times 100 = 90\%$$

Stage 4: Payload Execution

Banking credentials captured from infected users:

$$B = 5,000$$

Credential theft rate:

$$\frac{5,000}{22,500} \times 100 = 22.2\%$$

Stage 5: Covering Tracks & Exploitation

Compromised banking accounts used for fraudulent transactions:

$$F = 2,000$$

Exploitation success rate:

$$\frac{2,000}{5,000} \times 100 = 40\%$$

If the average fraudulent transaction was **£2,500** per account:

$$L = 2,000 \times 2,500$$

$$L = £5,000,000$$

Real-World Example

REAL-WORLD EXAMPLE

Back Orifice (1999) — marketed at the time as a remote-administration tool but widely used as a Trojan. Once running on a victim's Windows machine it gave the attacker full remote control, comparable to Windows Remote Desktop, entirely without the victim's knowledge.

Trojans in Practice

The book calls Trojans the most insidious network attack method — because the real weakness they exploit isn't technical, it's human trust.

Back Orifice & the Backdoor/Rootkit Pairing

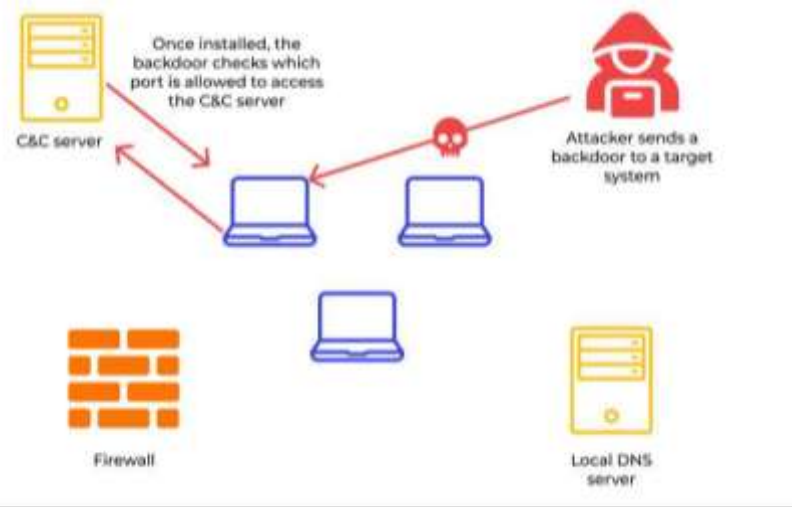
- A Trojan's real damage usually comes from what it installs next: a backdoor or rootkit that lets the attacker regain access later.
- A rootkit specifically installs after the initial attack and buries itself inside OS tools, making it extremely hard to detect.
- Back Orifice (since 1999) gives full remote control of the victim machine, functionally similar to Windows Remote Desktop — without the user's knowledge.
- Ironically, it is now marketed as a legitimate admin/remote-management tool rather than a hacking tool.
- Specific network ports are commonly associated with known Trojan programs, which is one reason port-monitoring matters.

Security Bytes: The Human Weak Link

WHY FIREWALLS AREN'T ENOUGH

Home firewall software is generally decent at spotting port-scanning attempts or shady connection attempts (e.g. on port 6667, a common IRC/botnet port). The real weakness is human behavior: firewalls often just prompt "Allow or Disallow?", and uninformed users click Allow without understanding what they're approving. Many Trojans also piggyback on standard, normal-looking ports, making malicious traffic hard for average users to tell apart from legitimate traffic. Where no corporate firewall policy handles this automatically, user education becomes the real control.

Backdoors



Any method that lets someone bypass normal authentication to access a system, application, or device — planted by malware, or sometimes built into a product from the start.

Types of Backdoors

Software/Application Backdoor

A hidden function or hardcoded account built into an app's own code, bypassing its normal login.

Hardware Backdoor

A hidden access mechanism built into a device's firmware or chip.

Web Shell Backdoor

A small malicious script uploaded to a web server, giving a persistent command interface through the website itself.

Cryptographic Backdoor

A deliberate weakness planted in an encryption algorithm or its implementation.

Supply-Chain Backdoor

Malicious code inserted into a legitimate vendor update, distributed to victims as a normal, trusted release.

How It Works

1. Delivery & Installation

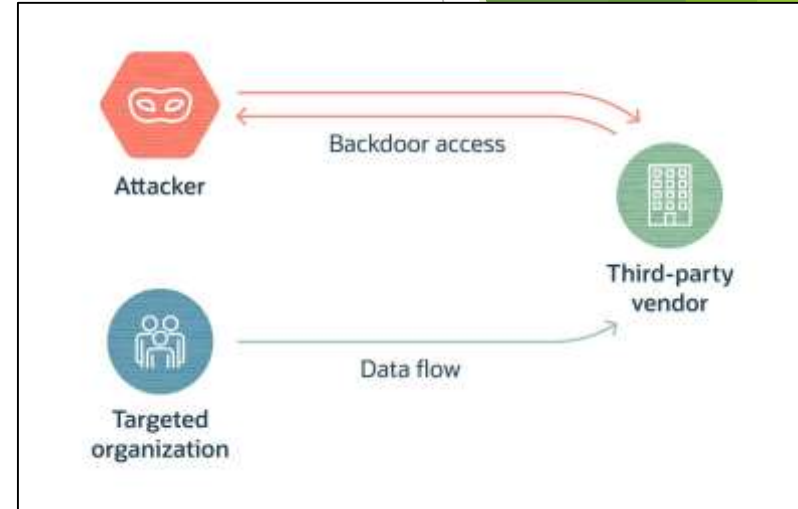
- **Action:** The attacker creates a deliberate, unauthorized entry point, often by exploiting a vulnerability or installing a "hidden door" within legitimate software.
- **Example:** A developer intentionally inserts a hidden login function into a company's custom software that allows them to bypass the standard authentication page using a secret "master" username.

2. Concealment

- **Action:** The backdoor is masked to avoid detection by security administrators or automated scanners.
- **Example:** The attacker names the malicious file `sys_helper.dll` and places it in a deep system directory, hoping it looks like a standard, harmless operating system component.

3. Persistence

- **Action:** The attacker ensures the backdoor remains active through system updates, reboots, or password changes.
- **Example:** The backdoor modifies a common system configuration file (like `.bashrc` on Linux or a Windows Registry Run key) to ensure it executes automatically every time the system boots up.



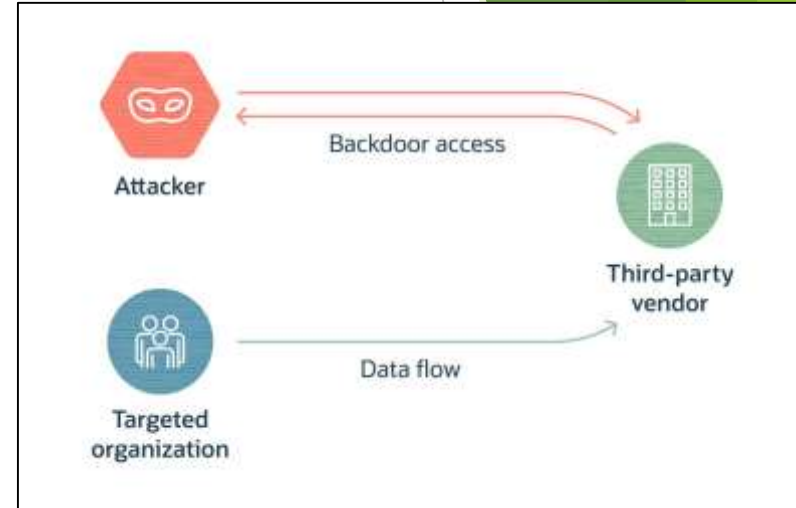
How It Works

4. Remote Access & Command

- **Action:** The attacker establishes a connection to the compromised system to execute commands or retrieve data remotely.
- **Example:** The attacker connects to the hidden port opened by the backdoor using a custom client, giving them a command-line interface to browse the server's files without needing a legitimate user account.

5. Maintaining Stealth & Avoiding Detection

- **Action:** The attacker modifies logs or system behavior to ensure their remote activities do not raise alerts.
- **Example:** The backdoor is programmed to automatically delete lines from the server's access logs that contain the attacker's IP address, preventing security teams from seeing who connected to the machine.



Example Scenario: DoublePulsar Backdoor

Victim System

Item	Value
Victim Server	Windows Server 2012
Server IP	192.168.1.20
SMB Port	445
Attacker IP	203.0.113.50
Vulnerability	MS17-010 (EternalBlue)

Stage	Action	Real-Life Example (DoublePulsar Backdoor)
1. Delivery & Installation	The attacker creates an unauthorized entry point.	The attacker scans the network and finds 192.168.1.20 with SMB port 445 exposed. Using the EternalBlue (MS17-010) exploit, the attacker installs the DoublePulsar backdoor in the server's memory without requiring valid login credentials.
2. Concealment	The backdoor hides itself from administrators.	DoublePulsar runs entirely in kernel memory instead of creating obvious files on disk, making it difficult for users and antivirus software to detect during routine file scans.
3. Persistence	The attacker ensures continued access.	After each successful compromise, the attacker can reinstall or maintain the backdoor and use it to deploy additional malware whenever the vulnerable service remains exposed.
4. Remote Access & Command	The attacker remotely controls the compromised system.	From 203.0.113.50 , the attacker sends specially crafted SMB commands to 192.168.1.20 . The backdoor executes the received instructions, such as loading malware, running programs, or collecting system information.
5. Maintaining Stealth & Avoiding Detection	The attacker minimizes signs of the compromise.	Because DoublePulsar communicates through SMB and resides mainly in memory, its activity is less obvious than a normal remote login. It was frequently used as a covert platform to deploy ransomware such as WannaCry while avoiding immediate detection.

Numerical Workflow

Assume:

Vulnerable servers scanned = **200,000**

Servers successfully infected with the DoublePulsar backdoor = **20,000**

Servers remotely controlled by attackers = **16,000**

Stage 1: Delivery & Installation

Vulnerable servers identified and targeted:

$$V = 200,000$$

Stage 2: Concealment

Servers where the DoublePulsar backdoor was successfully installed and hidden in kernel memory:

$$C = 20,000$$

Installation success rate:

$$\frac{20,000}{200,000} \times 100 = 10\%$$

Stage 3: Persistence

Servers where the backdoor remained active after installation:

$$P = 18,000$$

Persistence rate:

$$\frac{18,000}{20,000} \times 100 = 90\%$$

Stage 4: Remote Access & Command

Compromised servers that successfully received and executed remote commands from the attacker's C2 server:

$$R = 16,000$$

Remote command success rate:

$$\frac{16,000}{18,000} \times 100 = 88.9\%$$

Stage 5: Maintaining Stealth & Avoiding Detection

Servers that remained undetected while continuing malicious activities:

$$S = 14,000$$

Stealth success rate:

$$\frac{14,000}{16,000} \times 100 = 87.5\%$$

If **35%** of these stealthy compromised servers were later used to deploy ransomware (e.g., WannaCry):

$$W = 0.35 \times 14,000$$

$$W = 4,900 \text{ servers}$$

Result: 4,900 compromised servers were later used to deploy ransomware or other malicious payloads while the DoublePulsar backdoor remained hidden in kernel memory.

Real-World Example

REAL-WORLD EXAMPLE

The 2020 SolarWinds Orion supply-chain attack — attackers compromised the vendor's own build process and inserted a hidden backdoor (SUNBURST) into a routine, digitally signed update. Thousands of organizations installed it trusting its authenticity, unknowingly handing attackers backdoor access; the intrusion was later attributed to a Russian state-linked group.

Prevention Measures

- Rigorous code review and security testing to catch hidden backdoor functions before release.
- Verify software update authenticity (digital signatures, checksums) and vet vendor supply-chain security.
- Monitor for unexpected outbound connections or beaconing behavior.
- Apply least privilege so few accounts/services can open new listening ports or scheduled tasks.
- Run regular security audits/penetration tests specifically looking for undocumented access paths.
- Disable or remove vendor debug/maintenance accounts before a system goes into production.

Understanding Malware



What "Malware" Actually Means

- Network attacks generally happen for one of two reasons: stealing data for money, or pushing a political/social agenda.
- Targets are mostly organizations and their networked computers, but personal devices — phones, IoT gadgets — are fair game too.
- "Malware" is the umbrella term: viruses, worms, Trojans — any malicious software put into a network to help an attacker reach their goal.
- The lines between categories are getting blurry — modern malware often does multiple things at once instead of being "just a virus" or "just a worm".

How the Goal Has Shifted Over Time

Old Malware: Motive

Mostly wanted to destroy/corrupt data or crash a system — disruption for its own sake.

Modern Malware: Motive

Almost always about making money; cybercrime groups run teams of programmers writing malware with signatures antivirus doesn't recognize yet.

Old Malware: Targets

Targeted Windows/Linux and other traditional operating systems.

Modern Malware: Targets

Targets tablets, smartphones, and any connected device, not just traditional computers.

Security Bytes: SolarWinds & Silverfish

A LESSON FOR ETHICAL HACKERS

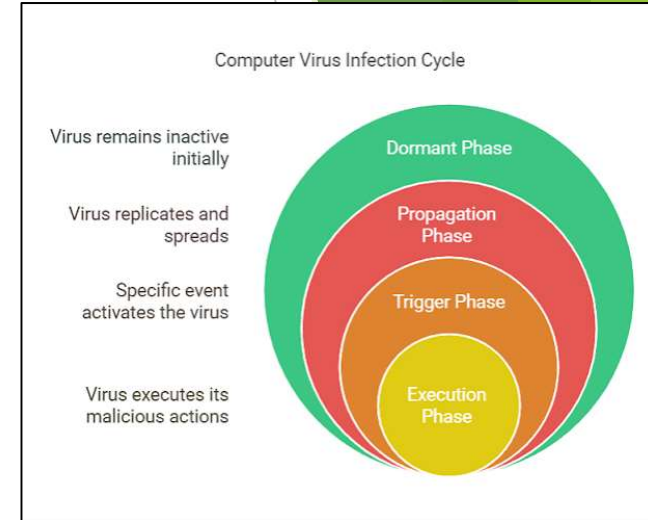
The 2020 SolarWinds attack was carried out by a group nicknamed Silverfish, who borrowed server infrastructure from another crime group called EvilCorp. There was no official nation attribution, but signs pointed toward Russia. The lesson: whatever security tool you learn to use, criminals are learning it too — which is exactly why organizations need ethical hackers.

Viruses

A program that attaches itself to a file or another program — that's the defining trait. A virus can't run or replicate on its own; it needs a host, the way flu needs a living body.

Phases of Virus

Phase	Description
1. Dormant Phase	The virus is idle, resides on the system, and remains hidden to avoid detection.
2. Propagation Phase	The virus replicates itself, placing copies into other programs or system areas.
3. Triggering Phase	A specific event (like a date, a number of replications, or a user action) occurs, which "wakes up" the virus.
4. Execution Phase	The virus releases its payload . This is the phase where the intended damage (or action) actually takes place.



Types of Viruses

File-Infector Virus

Attaches itself to an executable program, activating whenever that program runs.

Boot Sector Virus

Infects the disk's boot sector, loading before the operating system itself.

Macro Virus

Written as a macro inside apps like MS Office; fires automatically the moment a file is opened.

Polymorphic Virus

Rewrites its own code on each infection to dodge signature-based antivirus detection.

Resident Virus

Embeds itself in system memory, infecting files as they're opened even after the original carrier is gone.

Ransomware Virus

Locks up a system or its files until a ransom is paid — increasingly also stealing cloud credentials.

Real-World Example

REAL-WORLD EXAMPLE

Melissa (1999) — the most infamous macro virus ever. It triggered the moment a user opened an infected Word document, then automatically emailed itself to the first 50 contacts in that person's address book, spreading explosively through personal trust networks.

How It Works

1. Infection

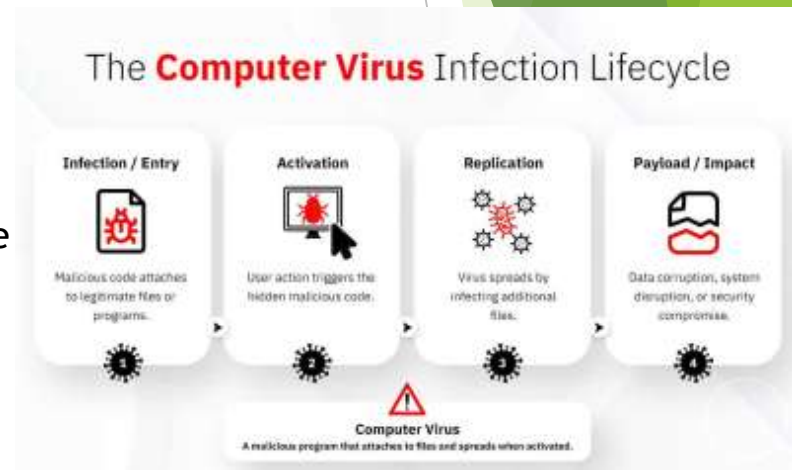
- **Action:** The virus attaches its malicious code to a legitimate host file or program, waiting to be executed.
- **Example:** A user downloads a shared document from an insecure drive; the virus embeds itself into the document's macro code.

2. Execution & Triggering

- **Action:** The user runs the infected program, activating the virus code.
- **Example:** When the user opens the infected document, the macro runs and silently triggers the virus's replication routine.

3. Replication

- **Action:** The virus copies itself into other healthy files or programs on the system.
- **Example:** The virus scans the local hard drive for all .exe files and appends its own malicious code to every one of them.



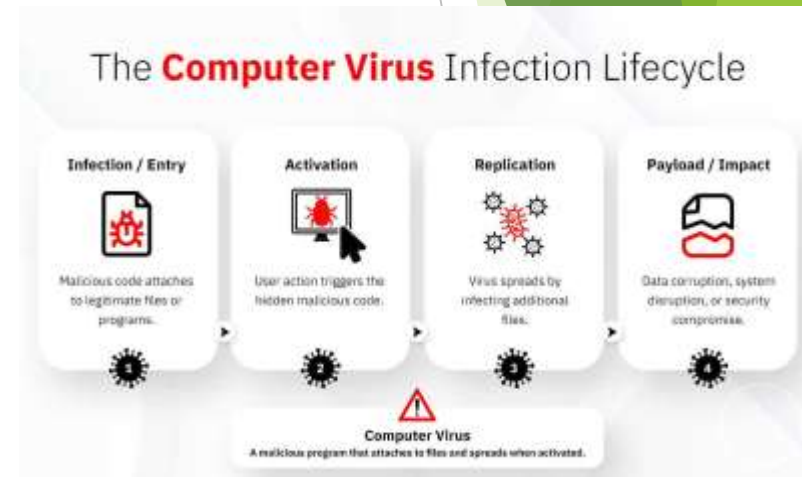
How It Works

4. Payload Activation

- **Action:** The virus performs its intended function, which is often destructive or disruptive.
- **Example:** On a specific date, the virus activates and begins deleting files or corrupting the user's photos and personal documents.

5. Propagation

- **Action:** The virus attempts to spread to other systems.
- **Example:** The virus detects a connected USB flash drive and immediately copies its infected files onto it, ensuring that when the drive is plugged into a different computer, the cycle starts over.



Example Scenario: Melissa Macro Virus (1999)

Victim System

Item	Value
Victim PC	Windows Computer
User Email	alice@company.com
Infected File	Important_Document.doc
USB Drive	*E:*
Network	192.168.1.25

Stage	Action	Real-Life Example (Melissa Virus)
1. Infection	The virus attaches itself to a legitimate file or program.	Alice receives an email with the attachment Important_Document.doc . The document contains a malicious Microsoft Word macro. She downloads the file to her computer (192.168.1.25).
2. Execution & Triggering	The infected file is opened, activating the virus.	Alice opens Important_Document.doc and enables macros. The hidden macro executes automatically and installs the Melissa virus in Microsoft Word without displaying any warning.
3. Replication	The virus copies itself into other files or applications.	Melissa infects the global Microsoft Word template (Normal.dot), causing every new Word document created on the computer to contain the malicious macro automatically.
4. Payload Activation	The virus performs its intended malicious activity.	Melissa accesses Microsoft Outlook and automatically sends the infected document to the first 50 contacts in Alice's address book, creating massive email traffic that slows or crashes corporate mail servers.
5. Propagation	The virus spreads to additional computers.	Bob receives the infected email from Alice, opens the attachment, enables macros, and his computer (192.168.1.40) becomes infected. The same process repeats, spreading the virus throughout the organization.

Mathematical Workflow

Assume:

Malicious Word documents delivered =
100,000, Users who enabled macros =
12,000, Systems successfully infected =
10,000

Stage 1: Infection

Malicious Microsoft Word documents
delivered:

$$D = 100,000$$

Stage 2: Execution & Triggering

Users who opened the document and
enabled macros:

$$E = 12,000$$

Execution rate:

$$\frac{12,000}{100,000} \times 100 = 12\%$$

Stage 3: Replication

Systems where the Melissa virus successfully
infected the **Normal.dot** template and
replicated itself:

$$R = 10,000$$

Replication success rate:

$$\frac{10,000}{12,000} \times 100 = 83.3\%$$

Stage 4: Payload Activation

Infected computers that automatically emailed
the malicious document to contacts:

$$P = 9,000$$

Payload activation rate:

$$\frac{9,000}{10,000} \times 100 = 90\%$$

Stage 5: Propagation

New computers infected after recipients
opened the infected attachment:

$$N = 18,000$$

Propagation rate:

$$\frac{18,000}{9,000} \times 100 = 200\%$$

If each infected computer sent the malicious
document to **50 contacts**, the total number of
phishing emails generated was:

$$M = 9,000 \times 50$$

$$M = 450,000 \text{ emails}$$

Result: 18,000 new computers became
infected through email propagation, while
450,000 infected emails were automatically
distributed, enabling the Melissa virus to spread
rapidly across organizations.

Prevention Measures

- Keep antivirus signature databases updated constantly, since unmatched signatures slip through.
- Layer in heuristic/behavioral detection to catch malware that hasn't been formally cataloged yet.
- Use sandboxing to test suspicious files in an isolated environment before they touch a real system.
- Block macros from auto-running by default in Office and similar macro-capable applications.
- Deploy antivirus at every layer — user devices, email servers, and network security appliances.

Macro Viruses: A Closer Look

A macro virus is technically a form of Trojan horse attack — a specific example of how a malicious payload gets delivered to an unsuspecting victim.

How Macro Viruses Operate

Stage	Action	Example
1. Infection & Delivery	The virus is embedded as a hidden script within a document and sent to the target.	Receiving an email with an attachment named Project_Plan.docm.
2. Triggering	The user opens the file and is tricked into enabling macro permissions.	Clicking the "Enable Content" button on the yellow security bar in Word.
3. Execution	The malicious script runs and interacts with the OS or other files.	The macro silently executes a shell command to download additional malware.
4. Replication	The virus copies itself into the application's global template.	The virus modifies Normal.dotm, infecting every new document you create.
5. Payload Activation	The virus performs its final objective (data theft, sabotage, etc.).	The macro emails your saved contacts or deletes local sensitive files.

Who Creates Viruses Today

FROM SKILLED PROGRAMMERS TO TUTORIAL SITES

It used to take skilled programmers who enjoyed the challenge. Today, even non-programmers can build one — step-by-step tutorial sites are freely available online. The book's advice for security professionals: get into the mindset of the creators, like an FBI profiler would, by reading what virus authors write about their own work. Key insight: hackers and malware authors aren't only motivated by money — a lot of it is about being first to exploit something, for prestige and bragging rights.

Notable Malware: A Highlight Reel

Notable Malware: A Highlight Reel

Ryuk (Ransomware)

Responsible for over a third of all ransomware attacks in 2020. Targets companies, hospitals, and government municipalities, encrypting critical files and typically demanding a multimillion-dollar ransom.

FormBook

A data-stealer/form-grabber family that steals clipboard contents, logs keystrokes, and steals data while browsing. Sold as "malware-as-a-service" on hacking forums, usually spread via spam email attachments.

CryptoLocker

Considered the "father" of modern ransomware. Estimated to have infected 250,000+ computers by 2016, locking files in an encrypted container and giving victims a countdown to pay before losing access.

Notable Malware: A Highlight Reel (cont.)

MalumPOS

Targets point-of-sale payment terminals; used in mid-2015 to hit hotel-chain POS systems, exfiltrating card numbers and magnetic-strip data. A newer strain shifted to stealing personal identifying info instead.

Carbanak

Spread via phishing emails aimed at financial institutions, carrying a Word document plus a malicious .cpl file. Once it confirms sufficient privileges, it opens a backdoor enabling fraudulent bank transfers and ATM fraud.

Gumblar

First seen March 2009, spread by mass-hacking websites via Adobe PDF/Flash vulnerabilities. Resurfaced in 2020 hitting WordPress/Drupal/Joomla sites, stealing FTP credentials and hijacking search results.

Notable Malware: A Highlight Reel (cont.)

Gpcode (a.k.a. PGPCoder)

Detected in 2008, still active as of 2020. Notable for using practically unbreakable 1024-bit asymmetric encryption to lock a victim's documents and hold them for ransom.

Ransomware & How Antivirus Detects It

No defense is foolproof — new viruses appear constantly, so nothing can guarantee 100% protection. Here's how ransomware has evolved, and the two core detection techniques antivirus relies on.

Ransomware: Then vs. Now / Detection Techniques

Ransomware: Then

Used to just lock local files on the infected device.

Ransomware: Now

Often also steals cloud storage credentials, locking victims out of their cloud accounts too, not just their local device.

Signature Matching

Compares a file's hash/code pattern against a database of known virus "fingerprints" — misses anything not yet cataloged.

Heuristic Analysis

Looks for suspicious behavior patterns typical of malware, catching things that haven't been formally identified yet.

How it Works

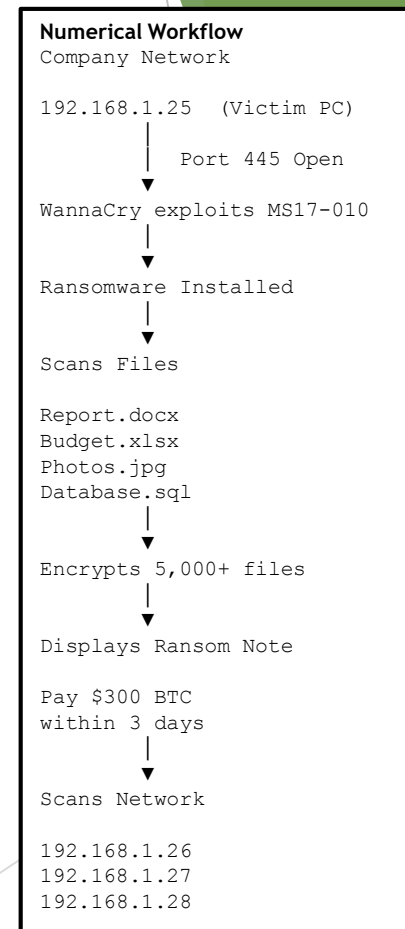
Stage	Action	Example
1. Infection & Delivery	The malware is delivered to the target system, often via phishing or exploit kits.	Opening an email attachment disguised as a "Urgent Tax Notice" that contains an encrypted payload.
2. Staging & Execution	The ransomware runs in the background, reaching out to the attacker's server for instructions.	The malware silently contacts a remote Command & Control server to download a unique encryption key.
3. Scanning & Encryption	The malware locates and encrypts target files (documents, photos, databases) across the system.	The ransomware begins encrypting all .docx, .pdf, and .jpg files on your hard drive and mapped network drives.
4. Extortion	The ransomware displays a notification informing the user of the lockout and demanding a ransom.	A popup appears on your desktop with a countdown timer, demanding payment in Bitcoin to receive the decryption key.
5. Post-Action/Persistence	The attacker may attempt to delete backups or spread to other machines on the network.	The ransomware attempts to locate and delete "Shadow Volume Copies" (Windows backups) to prevent you from restoring data for free.

Example Scenario: WannaCry Ransomware (2017)

Victim System

Item	Value
Victim PC	Windows 7 Computer
IP Address	192.168.1.25
SMB Port	445
Vulnerability	MS17-010 (EternalBlue)
Attacker Server	203.0.113.50

Stage	Action	Real-Life Example (WannaCry)
1. Infection & Delivery	Malware reaches the victim's computer.	Computer 192.168.1.25 has not installed Microsoft's MS17-010 security update. WannaCry scans the network, finds port 445 open, exploits the EternalBlue vulnerability, and infects the computer automatically without any user action.
2. Staging & Execution	The ransomware installs itself and begins running.	WannaCry copies its files into the Windows system directory, creates processes to execute the ransomware, and prepares the encryption routine. The malware starts automatically in the background.
3. Scanning & Encryption	The malware searches for valuable files and encrypts them.	WannaCry scans the computer and finds files such as Report.docx , Budget.xlsx , Photos.jpg , and Database.sql . Using strong encryption, it encrypts thousands of files and changes their extensions so they cannot be opened.
4. Extortion	The victim is informed that the files have been encrypted and is asked to pay a ransom.	A red warning screen appears stating that the files have been encrypted. The victim is instructed to pay \$300 in Bitcoin within 3 days , after which the ransom increases to \$600 . A countdown timer is displayed.
5. Post-Action / Persistence	The malware attempts to prevent recovery and spread further.	WannaCry searches the local network for other vulnerable Windows computers (e.g., 192.168.1.26 , 192.168.1.27 , 192.168.1.28) and infects them using the same EternalBlue exploit. On affected systems, recovery options are also hindered, making restoration more difficult.



Mathematical Workflow

Assume:

Vulnerable computers scanned = **500,000**, Computers successfully infected = **50,000**, Computers with encrypted files = **45,000**

Stage 1: Infection & Delivery

Vulnerable computers identified and targeted:

$$V = 500,000$$

Stage 2: Staging & Execution

Computers where WannaCry was successfully installed using the **EternalBlue (MS17-010)** exploit:

$$I = 50,000$$

Installation success rate:

$$\frac{50,000}{500,000} \times 100 = 10\%$$

Stage 3: Scanning & Encryption

Computers whose files were successfully encrypted:

$$E = 45,000$$

Encryption success rate:

$$\frac{45,000}{50,000} \times 100 = 90\%$$

Stage 4: Extortion

Victims who paid the ransom demand:

$$R = 9,000$$

Payment rate:

$$\frac{9,000}{45,000} \times 100 = 20\%$$

If each victim paid **£300** in ransom:

$$L = 9,000 \times 300$$

$$L = \text{£}2,700,000$$

Stage 5: Post-Action / Persistence

Additional computers infected through lateral movement across the network:

$$P = 30,000$$

Propagation rate:

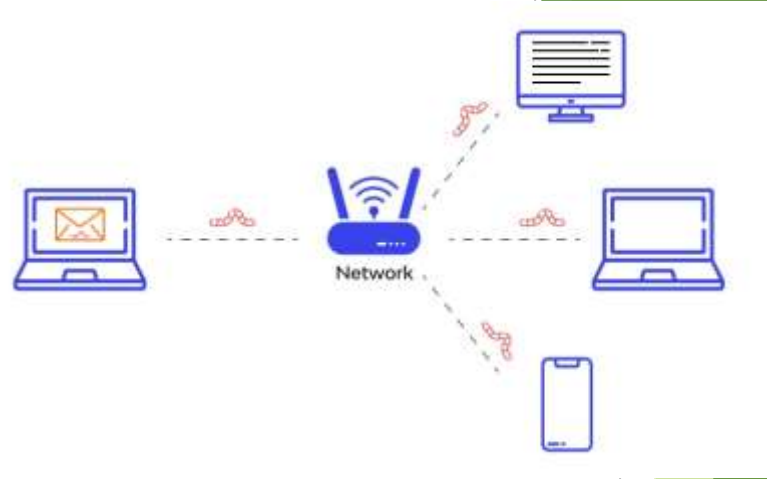
$$\frac{30,000}{45,000} \times 100 = 66.7\%$$

Result: 30,000 additional computers were infected through network propagation, while **9,000 victims** paid the ransom, resulting in an estimated **£2,700,000** in ransom payments.

Beyond Antivirus

- Symantec Endpoint Protection (SEP) lets admins push signature updates automatically to every client machine in an organization.
- Network security devices watch traffic across the whole network and intercept malware before it ever reaches a user.
- Sandboxing runs a suspicious file in an isolated environment where it can't write to the real hard drive, to safely test whether it's malicious.

Worms



Unlike a virus, a worm doesn't need to attach to a host file — it replicates and spreads on its own, "crawling" across a network hunting for other devices to infect.

Phases of Worms

Phase	Description
Information Gathering	The worm scans networks to identify active systems, open ports, and vulnerable targets.
Infection	The worm exploits system vulnerabilities (like buffer overflows) to breach the target machine without user interaction.
Payload	The programmed malicious action executes (e.g., creating backdoors, dropping ransomware, or stealing data).
Network Propagation	The worm replicates itself and generates new IP addresses to attack, restarting the cycle on the newly infected machine.

Types of Worms

Network Worm

Spreads directly machine-to-machine by exploiting a network service vulnerability.

Email Worm

Spreads via mass auto-generated emails or malicious attachments.

Cryptoworm

Combines self-propagation with a ransomware payload that detonates after spreading.

Botnet Worm

Spreads specifically to recruit infected machines into a remotely controlled botnet.

Real-World Example

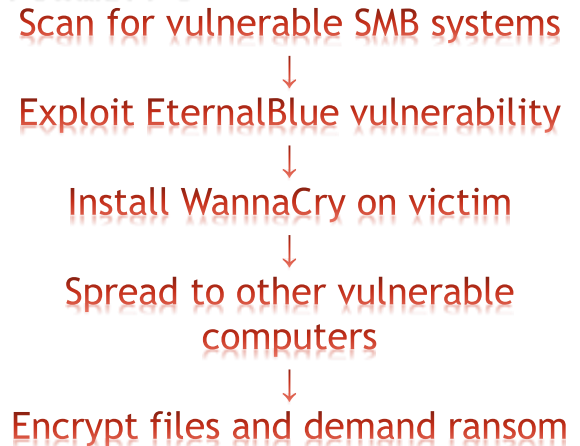
REAL-WORLD EXAMPLE

WannaCry (2017) — a ransomware cryptoworm that exploited a vulnerability in Windows' SMB file-sharing protocol to spread machine-to-machine, then detonated its ransomware payload, infecting 230,000+ computers across roughly 150 countries within a single day.

How It Works

Stage	Action	Example
1. Scanning	The worm searches the network for other computers or devices with known security vulnerabilities.	The worm scans local IP ranges to find systems that have not patched a specific "Remote Code Execution" bug.
2. Exploitation	The worm automatically attacks the identified vulnerability to gain unauthorized access.	It sends a specially crafted network packet to a vulnerable system, bypassing the need for a user to log in.
3. Installation	The worm copies its own executable code to the target system and launches itself.	Once the exploit succeeds, the worm downloads its main body into the target's memory and starts running as a background process.
4. Replication	The worm uses the compromised system as a new launchpad to scan and infect other machines.	The newly infected computer begins scanning the rest of the internal company network for more targets.
5. Payload Activation	The worm executes its primary objective, which may be disruptive or malicious.	The worm triggers a payload that opens a backdoor for an attacker, installs a botnet agent, or simply causes massive network congestion.

EXAMPLE -



Mathematical Workflow

Assume:

Vulnerable computers scanned = **1,000,000**

Computers successfully exploited = **80,000**

Computers infected with the worm = **75,000**

Stage 1: Scanning

Computers scanned for known vulnerabilities:

$$S = 1,000,000$$

Stage 2: Exploitation

Computers successfully exploited through the security vulnerability:

$$E = 80,000$$

Exploitation rate:

$$\frac{80,000}{1,000,000} \times 100 = 8\%$$

Stage 3: Installation

Computers where the worm successfully installed and began executing:

$$I = 75,000$$

Installation success rate:

$$\frac{75,000}{80,000} \times 100 = 93.75\%$$

Stage 4: Replication

New computers infected as the worm spread automatically across the network:

$$R = 225,000$$

Replication rate:

$$\frac{225,000}{75,000} \times 100 = 300\%$$

Stage 5: Payload Activation

Infected computers that executed the worm's malicious payload (e.g., ransomware, botnet installation, or backdoor creation):

$$P = 180,000$$

Payload activation rate:

$$\frac{180,000}{225,000} \times 100 = 80\%$$

If **25%** of the payload-activated systems had critical business files encrypted:

$$C = 0.25 \times 180,000$$

$$C = 45,000 \text{ computers}$$

Result: 45,000 computers suffered critical file encryption or other severe damage after the worm propagated automatically across the network.

Prevention Measures

- Patch network-service vulnerabilities promptly — worms almost always exploit a known, unpatched flaw.
- Segment networks so a worm can't freely scan and spread across the entire organization.
- Disable unnecessary network services/ports that worms commonly target.
- Deploy intrusion detection to catch the scanning behavior worms rely on to find new victims.
- Maintain offline backups, since worm cleanup often means full reinstalls plus extra IT staff time.

Notable Worms

Worms have collectively cost businesses billions — not just from the attack itself, but downtime, data recovery, reinstalling systems, and hiring extra IT staff to clean up.

Notable Worms

WannaCry (2017)

A ransomware cryptoworm that exploited a Windows SMB vulnerability to spread machine-to-machine. Infected 230,000+ computers across ~150 countries in a single day.

Flame (a.k.a. KyWiper)

Discovered May 2012, often called the most complex malware ever built. Could spy via microphone/webcam, log keystrokes, and capture screens on both local and remote machines.

Stuxnet

Found in 2010 on industrial control systems at an Iranian nuclear facility. Believed to have entered via USB; physically manipulated uranium refinement centrifuges until they failed.

Notable Worms (cont.)

Duqu

Detected October 2011, structurally similar to Stuxnet but aimed at stealing data rather than physical sabotage. Mostly targeted government agencies in Europe and the Middle East.

Storm

Detected January 2007. Spread via mass auto-generated emails and became a huge botnet, estimated to have infected millions of systems.

Waledac

An email worm that harvests and forwards stolen passwords, spreading via an attachment named eCard.exe. One variant used geo-IP lookup to localize a fake "dirty bomb" news story near the victim.

Notable Worms (cont.)

Conficker

Detected late 2008. A botnet worm exploiting a Microsoft network service vulnerability, updating itself dynamically; detectable with a standard port scanner plus a Conficker-specific plug-in.

Slammer

Detected 2003. Reportedly knocked out 13,000+ ATMs of one of the largest banks in America by infecting the database servers on that bank's network.

Spyware & Adware

Two malware families that blur together — both quietly collect information about you, but they differ in how visible they choose to be.

Spyware

What is Spyware?

Spyware is a type of malicious software designed to secretly **monitor and record** your digital activity. Unlike ransomware (which makes itself known) or viruses (which often try to disrupt your system), spyware's primary goal is to remain invisible while it siphons off your personal information.

What does it do?

- Spyware operates as a silent observer. Its primary functions include:
- **Keystroke Logging:** Recording every character you type, including passwords, emails, and credit card numbers.
- **Browsing Surveillance:** Tracking every website you visit, your search history, and the files you download.
- **System Profiling:** Gathering information about your device, your installed software, and your network configuration.
- **Resource Hijacking:** Accessing your microphone, webcam, or screenshots to collect visual and audio data.
- **Data Exfiltration:** Regularly transmitting this collected data to a remote server owned by an attacker, who then uses it for identity theft, targeted phishing, or selling it to third-party data brokers.

Adware

What is Adware?

Adware (short for **advertising-supported software**) is a type of program that automatically displays, plays, or downloads advertisements to a computer or device. While some adware is legitimate—used by developers to monetize free software—**malicious adware** installs itself without your clear consent and often engages in deceptive practices to generate revenue for the attacker.

What does it do?

- Adware is designed to force ads onto your screen and track your habits to make those ads more "relevant" (and therefore more profitable). Its functions include:
- **Forced Displays:** Injecting unwanted banners, pop-ups, and full-page ads into your browser or desktop, even when you aren't using a web browser.
- **Search Hijacking:** Redirecting your browser's search queries to a different, often suspicious, search engine designed to serve more ads.
- **Behavioral Tracking:** Monitoring your browsing habits, shopping interests, and clicks to build a detailed advertising profile on you.
- **Performance Degradation:** Consuming system resources (CPU, RAM, and bandwidth) to load constant advertisements, which slows down your computer and drains battery life.
- **Aggressive Bundling:** Using "sneaky" installation techniques (like being pre-checked in the settings of other software installers) to ensure it gets onto your machine.

Defending Against Malware

Where Antivirus Should Live

- On every user's computer, obviously.
- Also on servers — especially email servers — to kill malware before it ever reaches a user's inbox.
- At the hardware level too, like Unified Threat Management (UTM) devices protecting the whole network.

Educating Your Users

- Structured training for all employees and management — mandatory at many U.S. government agencies, increasingly copied by private companies.
- Monthly security update emails to keep staff aware of the latest circulating threats.
- Simulated phishing campaigns — employees who click a bad link get routed to training, not punishment.
- Application whitelisting — only pre-approved programs are allowed to run at all, so unlisted malware simply can't execute.
- Keep virus signatures updated the moment vendors release them, centralized from a corporate server rather than relying on individual diligence.
- Firewalls — many antivirus vendors bundle software firewalls for home/small-business users; blocking known malicious ports (e.g. UDP 8998 used by W32/Sobig.F) stops specific attacks.
- Disable unnecessary default services (e.g. Telnet) that are rarely needed and vulnerable to external attacks.

Avoiding Fear Tactics

AWARENESS, NOT FEAR

Some security professionals exploit user ignorance with fear tactics — scaring users to drum up business for security testing services. The book calls this unethical and against the OSSTMM's Rules of Engagement, which explicitly states that fear, uncertainty, and doubt (FUD) must not be used in sales or marketing for security testing services. The better approach: promote awareness, not fear, and build on what users already understand.

Sniffers

Tools that capture and inspect network traffic as it passes by, letting an attacker (or a defender) see the raw data flowing across a network segment.

Types of Sniffers

Passive Sniffer

Silently listens on a shared network segment (e.g. a hub) without injecting any traffic.

Active Sniffer

Manipulates the network (e.g. ARP spoofing) to redirect switched traffic through the attacker's machine.

Wireless Sniffer

Captures Wi-Fi traffic over the air, often targeting unencrypted or weakly encrypted networks.

Protocol/Credential Sniffer

Targets specific unencrypted protocols (FTP, Telnet, HTTP) to pull out plaintext credentials.

How It Works

1. Preparation & Positioning

The attacker must be in a position where they can "see" the network traffic. This usually means being on the same local network as the target.

- Action: The attacker gains access to the local area network (LAN) via Wi-Fi or by plugging a device into a physical switch.
- Example: An attacker sits in a coffee shop and joins the public "Free_Guest_WiFi" network, which is the same network used by other customers.

2. Interception (The "Sniff")

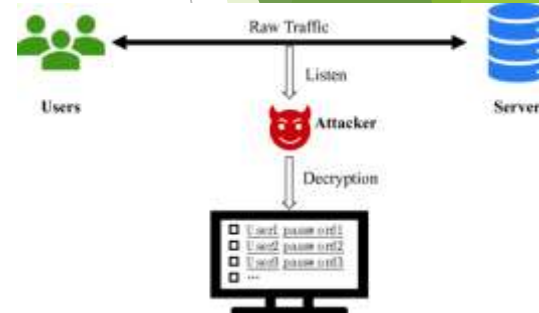
The attacker needs to force traffic to flow through their device, even if it is not addressed to them.

- Action: The attacker uses techniques like ARP Spoofing (telling the network router that the attacker's computer is actually the target victim's computer). This causes the victim's data to be sent to the attacker first.
- Example: The attacker uses a tool like *Ettercap* to send "spoofed" messages to the coffee shop's router, tricking it into routing all traffic from a nearby victim's laptop through the attacker's device.

3. Packet Capture

Once the traffic is flowing through the attacker's device, the sniffer software begins recording it.

- Action: The software puts the network interface card (NIC) into Promiscuous Mode, allowing it to capture all data packets regardless of the intended destination.
- Example: The sniffer tool, such as *Wireshark* or *Tcpdump*, begins writing every data packet intercepted to a capture file on the attacker's hard drive.



How It Works

4. Data Extraction & Decoding

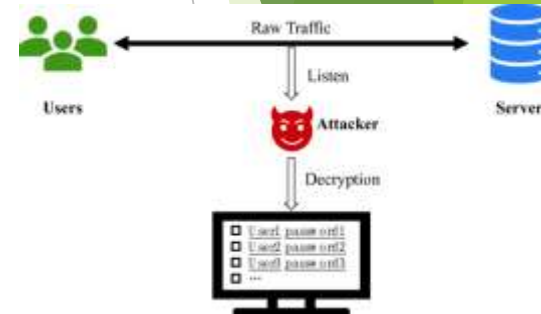
The captured data is raw and chaotic. The attacker must filter and reassemble the packets to find useful information.

- **Action:** The attacker uses protocol analysis to filter out "noise" (background traffic) and look for specific data types like HTTP requests, login forms, or email protocols.
- **Example:** The attacker filters the captured data to look for "POST" requests. They find a packet containing a plain-text username and password being sent to a non-secure website (using `http://` instead of `https://`).

5. Exploitation

The attacker uses the harvested information to gain unauthorized access.

- **Action:** Using the stolen credentials or session data to impersonate the victim.
- **Example:** The attacker uses the stolen username and password to log into the victim's personal email account from their own device, having successfully "sniffed" the credentials as they were typed in the coffee shop.



Example Scenario: Packet Sniffing on a Public Wi-Fi Network

Network Details

Device	IP Address
Wi-Fi Router (Gateway)	192.168.1.1
Victim Laptop	192.168.1.25
Attacker Laptop	192.168.1.30
Website	http://example-login.com

1. Preparation & Positioning

Action:

The attacker connects to the same public Wi-Fi network as the victim.

Example:

- Coffee shop Wi-Fi: **Free_Guest_WiFi**
- Router IP: **192.168.1.1**
- Victim receives **192.168.1.25**
- Attacker receives **192.168.1.30**

At this point, the attacker is simply another user on the same network.

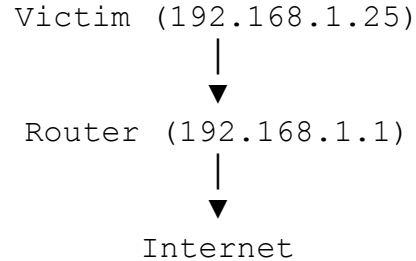
2. Interception

Action:

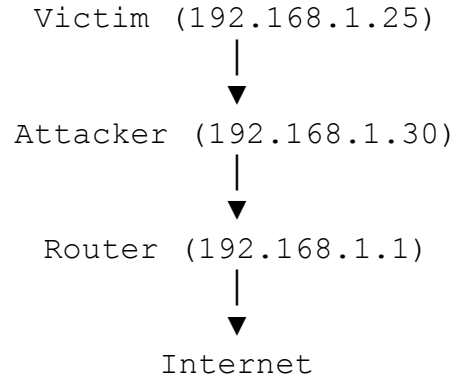
The attacker manipulates local network communication so traffic between the victim and the router passes through the attacker's device before reaching its destination.

Example:

Instead of the normal communication:



Traffic is redirected to:



The victim continues browsing normally without noticing any difference.

3. Packet Capture

Action:

The attacker's network analyzer records the packets passing through their device.

Example:

Suppose the victim visits:

`http://example-login.com`

Captured packets might include:

```
Source IP      : 192.168.1.25
Destination IP : 93.184.216.34
Protocol       : HTTP
Method         : POST
```

Every packet exchanged between the victim and the website is recorded.

4. Data Extraction & Decoding

Action:

The attacker filters the captured packets to identify useful information, such as unencrypted login requests.

Example:

A captured HTTP request may contain:

```
POST /login HTTP/1.1
```

```
Username = alice123
```

```
Password = MyPassword123
```

Because the website uses **HTTP** instead of **HTTPS**, the login data is transmitted in plain text and can be read directly from the captured packets.

5. Exploitation

Action:

The attacker attempts to use the stolen credentials to access the victim's account.

Example:

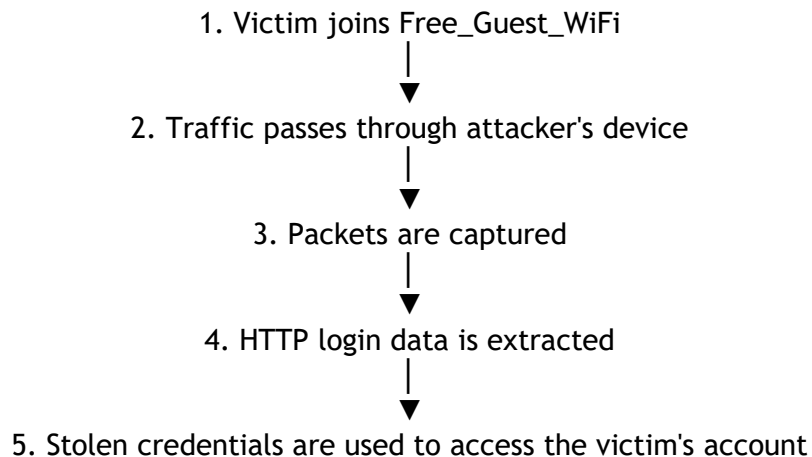
Using the captured credentials:

Username: `alice123`

Password: `MyPassword123`

The attacker logs into **example-login.com** as the victim and gains unauthorized access to the account.

Complete Workflow



Real-World Example

REAL-WORLD EXAMPLE

An attacker on a public coffee-shop Wi-Fi network runs a wireless sniffer in promiscuous mode, capturing unencrypted HTTP login traffic from nearby customers and extracting plaintext usernames and passwords straight out of the captured packets.

Prevention Measures

- Force all network gear and applications to communicate only over encrypted protocols with valid, trusted certificates.
- Use switched networks with port security to make passive sniffing harder.
- Deploy ARP-spoofing detection to catch active sniffing attempts.
- Use a VPN on untrusted networks (public Wi-Fi) to encrypt traffic end-to-end.
- Disable legacy plaintext protocols (Telnet, FTP) in favor of their encrypted equivalents.

Eavesdropping & Man-in-the-Middle

Eavesdropping

PASSIVE LISTENING

An attacker listens in on unencrypted network traffic to intercept sensitive data or credentials — usually via packet-sniffing tools like tcpdump and Wireshark, saved as .pcap files and later combed through for useful data. Defense: force all network gear and applications to communicate only over encrypted protocols with valid, trusted certificates.



Eavesdropping Working

Stage	Action	Example
1. Positioning	The attacker gains access to the communication medium (network or physical path).	An attacker joins a public, unencrypted Wi-Fi network at an airport.
2. Interception	The attacker places their device in a mode to capture data packets passing by.	Using a tool like <i>Wireshark</i> to capture all unencrypted traffic on the local network.
3. Filtering	The attacker separates useful data from background network noise.	Filtering the captured data to specifically look for "GET" or "POST" web requests.
4. Analysis	The attacker reassembles the intercepted packets to read the sensitive content.	Extracting a plain-text password sent over an unencrypted website (http://).
5. Exploitation	The attacker uses the intercepted information for malicious purposes.	Using the stolen password to log into the victim's social media



Man-in-the-Middle (MitM): Active Interception

While traditional eavesdropping is passive (just listening), a MitM attack is active because the attacker controls the communication flow.

Here is how the two concepts are related in points:

- **From Passive to Active:** Traditional eavesdropping involves just "listening" to the data passing by (like a sniffer). In a MitM attack, the attacker steps into the path to **actively control** the data stream.
- **The "Middle" Position:** Just like in eavesdropping, the attacker must position themselves between the sender and the receiver. However, the MitM attacker ensures the data is forced through their device, whereas an eavesdropper just "taps" into the wire or wireless signal.
- **Real-Time Manipulation:** While an eavesdropper records data to analyze later, a MitM attacker can **intercept and modify** that data in real-time. For example, they can eavesdrop on your email, change the text, and then forward the modified version to the recipient without either party knowing.
- **Breaking Encryption:** Eavesdropping often fails if the data is encrypted (e.g., HTTPS). A MitM attacker actively works to **break or bypass that encryption** (e.g., using "SSL Stripping" or presenting a fake certificate) so they can eavesdrop on what would otherwise be secure traffic.
- **Combined Goal:** The ultimate goal of both is the same: to gain unauthorized access to information. MitM is simply a more advanced, invasive version of the eavesdropping concept.

Eavesdropping Vs Man in the Middle

Feature	Eavesdropping	Man-in-the-Middle
Activity Level	Passive (Listening/Recording)	Active (Intercepting/Modifying)
Visibility	Nearly invisible to the target	Can be invisible or disrupt connections
Data Integrity	Data remains unchanged	Data can be altered or injected
Encryption	Blocked by strong encryption	Attacker attempts to bypass encryption
Tool Example	Network Sniffer (Wireshark)	Proxy/Spoofing Tool (Burp Suite/Ettercap)

Denial of Service (DoS / DDoS)

A DoS attack blocks legitimate users from accessing network resources. Unlike most attacks, the goal isn't stealing data — it's crippling access.

Types of DoS/DDoS

Ping of Death

Crafts an oversized ICMP packet that crashes the target when it's reassembled — mostly a 1990s relic now.

SYN Flood

Opens huge numbers of half-finished TCP connections, exhausting the target's connection table.

UDP Flood

Floods the target with UDP packets, forcing it to waste resources checking for listening applications.

Application-Layer DoS

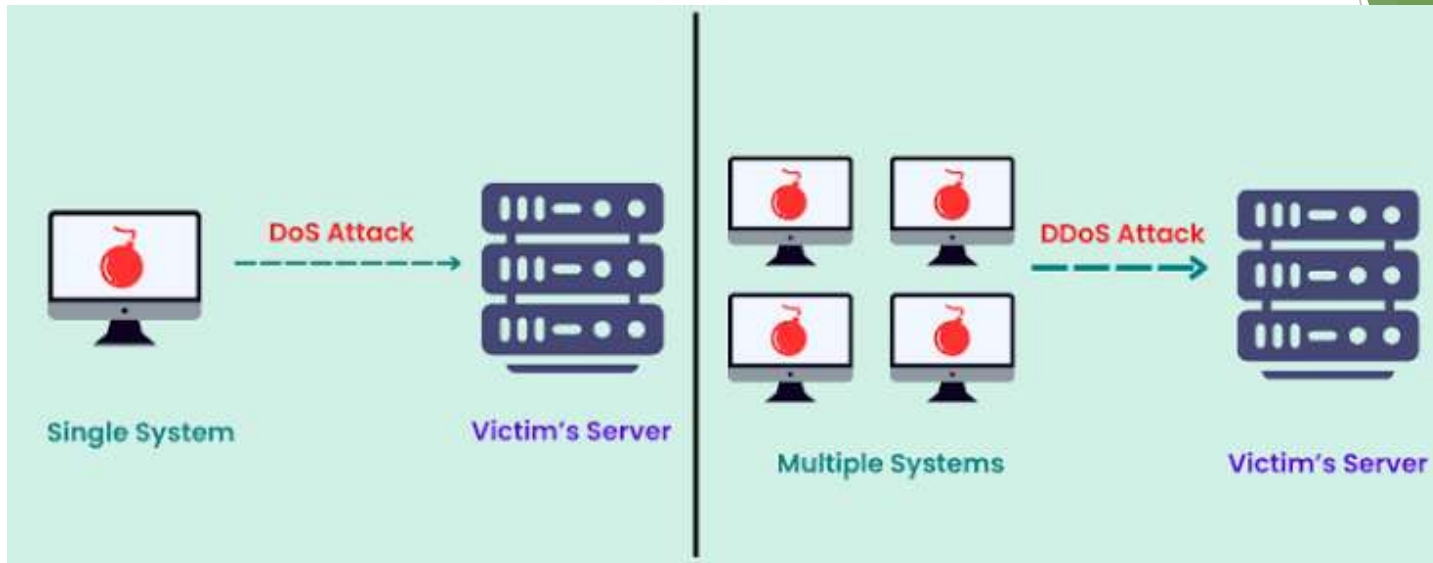
Overwhelms a specific application (e.g. a login form) rather than the raw network link.

DDoS (Distributed)

Same goal as DoS, but launched from thousands of compromised "zombie" machines in a botnet at once.

How It Works (DOS and DDOS)

Stage	DoS	DDoS
1. Preparation	Attacker targets a specific server or IP address.	Attacker infects thousands of devices to create a "botnet."
2. Triggering	Attacker floods the target with traffic from their one machine.	Attacker sends a "command" to the botnet to start attacking.
3. Execution	The target's system slows down or crashes under the strain.	Thousands of bots flood the target simultaneously from different locations.
4. Impact	The target becomes inaccessible to legitimate users.	The target's infrastructure collapses, causing a total blackout.



DoS Attack: This part of the diagram shows a **single system** acting as the source of an attack directed at a **victim's server**. Because it originates from one location, it is generally easier to identify and block.

DDoS Attack: This side illustrates a **DDoS attack**, where **multiple systems** (often referred to as a botnet) coordinate to flood a single **victim's server** with traffic simultaneously. By attacking from many different sources at once, the traffic becomes much harder to mitigate.

DOS Vs DDOS

Feature	DoS (Denial of Service)	DDoS (Distributed Denial of Service)
Source	Single source (one machine).	Multiple sources (a "botnet").
Ease of Defense	Easier to block by filtering one IP address.	Extremely difficult to block all attacking IPs.
Speed/Scale	Limited bandwidth/power.	Massive volume; can take down large services.
Visibility	Attacker's origin is often easy to trace.	Origin is hidden behind thousands of compromised devices.
Primary Goal	Exhaust resources of one target.	Overwhelm target via sheer volume of traffic.

Example(DOS and DDOS)

DoS Example: An attacker uses a simple script on their laptop to repeatedly send "SYN" packets to a small business's web server, filling its connection table so no real customers can connect.

DDoS Example: A large-scale botnet consisting of 50,000 infected smart home devices (IoT) around the world is instructed to send millions of requests per second to a major banking website. Because the requests come from 50,000 different IPs, the bank's security cannot simply block one IP, and their server resources are completely exhausted.

Real-World Example

REAL-WORLD EXAMPLE

The 2007 Estonia DDoS attack is one of the most studied DDoS incidents ever — it took down government sites, banks, and financial institutions, with malicious traffic sourced globally, including from the U.S. and Canada.

Prevention Measures

- Deploy DDoS mitigation/scrubbing services that absorb and filter flood traffic before it reaches the target.
- Use rate limiting and connection-timeout tuning to blunt SYN/UDP flood techniques.
- Maintain excess bandwidth and load-balanced infrastructure to absorb sudden spikes.
- Monitor for a Dark DDoS pattern — treat any large flood as a possible smokescreen for a second, quieter attack.
- Keep an incident response plan specifically for availability attacks, since DoS defense differs from data-theft defense.

Buffer Overflow Attacks

Dr. Indu Singh CSE DTU

The Core Mechanism

WHEN CODE DOESN'T ENFORCE ITS OWN LIMITS

A buffer overflow happens when poorly written code fails to enforce a fixed memory-space limit for a variable. Example logic: a variable is defined to hold 64 bytes, but the program lets data get written past that boundary without erroring out or blocking it — that overflow is the vulnerability. The attacker doesn't just fill the overflow with junk data — they fill it with executable code, which the OS then runs, typically to elevate the attacker's permissions to administrator level, or to create a service granting remote access to the target system.

Table 3-4: Notable Buffer Overflow Vulnerabilities

Cisco ASA IKE (CVE-2016-1287)

A specially crafted packet sent to an affected Cisco ASA device could grant full administrative privileges, remotely over the internet if the device sits on a network perimeter.

GHOST

Discovered by Qualys researchers, made global headlines. Under the right conditions, allowed remote administrative access with zero credentials, rooted in a glibc weakness.

PAN-OS (2020)

Palo Alto Networks patched a buffer overflow in PAN-OS, running on many next-gen firewalls; a remote, unauthenticated attacker could disrupt processes or execute arbitrary code.

Table 3-4: Notable Buffer Overflow Vulnerabilities (cont.)

StageFright (Android, CVE-2015-1538)

Proof buffer overflows aren't just a desktop problem — a specially crafted MMS message could trigger remote code execution with zero user interaction.

VPN Products (2020)

NSA issued alerts about buffer overflow flaws in Pulse Secure, Palo Alto GlobalProtect, and Fortinet FortiGate, allowing arbitrary file downloads and remote code execution.

Windows Server Service (MS08-067)

Let attackers run arbitrary code in memory — notable because this exact vulnerability is what let the Conficker worm spread.

Why This Keeps Happening

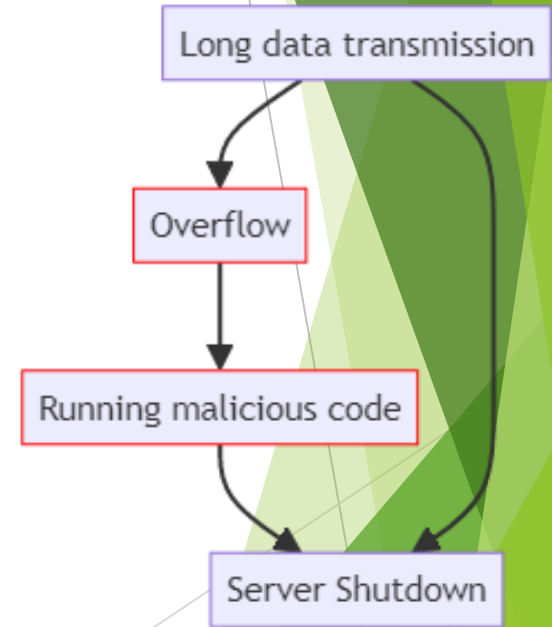
- Most programmers weren't trained with security in mind; code was historically written for speed, efficiency, and usability, not security.
- DevSecOps now exists specifically to bake security into development from the start.
- Supporting initiatives include OWASP (Open Web Application Security Project) and BSIMM (Building Security In Maturity Model).
- Microsoft, for example, now rewards programmers whose code doesn't later turn up as a vulnerability.

Buffer Overflow Attack Types

Attack Type	Target Memory Area	Mechanism	Primary Goal
Stack-Based	Stack	Overflows a local buffer to overwrite the "Return Address."	Redirect program flow to malicious code.
Heap-Based	Heap	Overflows a dynamically allocated buffer to corrupt memory structures.	Hijack memory pointers or corrupt application logic.
Integer Overflow	N/A (Calculation)	Provides input that causes a mathematical "wrap-around."	Force the program to allocate a buffer too small for the data.
Off-by-One	Boundary of Buffer	Writes exactly one byte past the intended memory limit.	Manipulate control structures using minimal corruption.

How it Works

Step	Action	Description
1. Input	Attacker provides oversized input	The application receives more data than its memory buffer was designed to store.
2. Overflow	Data spills into adjacent memory	The input overwrites surrounding memory addresses (the "overflow").
3. Hijack	Control is redirected	The overwritten memory typically contains a "Return Address" that tells the CPU where to go next.
4. Execution	Malicious code runs	The CPU is forced to follow the hijacked path, executing the attacker's code instead of the legitimate program.



Example

Suppose a program allocates a buffer of **8 bytes**.

The buffer occupies addresses **100-107**.

Step 1: Normal Input

Input:

ABCDEFGH

Length:

Input Size = 8 bytes

Buffer Size:

Buffer Size = 8 bytes

Since

$$8 \leq 8$$

everything fits perfectly.

The program executes normally.

Address	Stored Value
100	A
101	B
102	C
103	D
104	E
105	F
106	G
107	H
108	Return Address = 5000

Memory Address	Value
100	A
101	B
102	C
103	D
104	E
105	F
106	G
107	H
108	Return Address = 5000
109	Next Memory

Step 2: Oversized Input

Now suppose the attacker sends

ABCDEFGHIJKLMNOP

Length:

Input Size = 16 bytes

Buffer Size:

Buffer Size = 8 bytes

Overflow amount:

$16 - 8 = 8$ bytes

Those extra **8 bytes** overwrite the memory after the buffer.

The original return address at **108** has now been overwritten

Address	Stored Value
100	A
101	B
102	C
103	D
104	E
105	F
106	G
107	H
108	I
109	J
110	K
111	L
112	M
113	N
114	O
115	P

Step 3: Hijack

Originally,

Return Address = 5000

After overflow,

Return Address = 9000

So instead of

CPU → Address 5000 (legitimate code)

the CPU executes

CPU → Address 9000 (attacker-controlled code)

Mathematical Summary

Let

• Buffer size = B

• Input size = I

Overflow occurs when

$$I > B$$

The number of overflowing bytes is

$$\text{Overflow} = I - B$$

Example

$$B = 8, I = 16$$

Therefore,

$$\text{Overflow} = 16 - 8 = 8 \text{ bytes}$$

These **8 extra bytes** overwrite adjacent memory, which may include the **return address**, allowing the program's execution flow to be redirected.

Session Hijacking

An attacker takes over an already-authenticated session between a user and a system, tricking both legitimate parties into believing the attacker is the other party.

Types of Session Hijacking

Session Sniffing

Captures a session token/cookie directly off unencrypted network traffic.

Cross-Site Scripting (XSS) Hijack

Injects script into a trusted page to steal the victim's session cookie.

Session Fixation

Tricks the victim into using a session ID the attacker already knows before they even log in.

Man-in-the-Middle Hijack

Intercepts and relays the session live, riding along as both parties communicate.

Session Side-Jacking

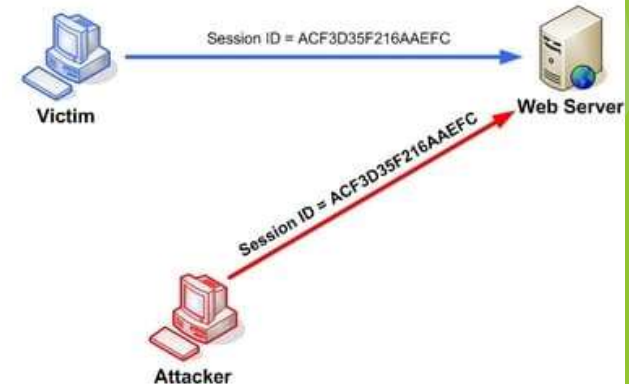
Sniffs traffic on an unsecured network (e.g. public Wi-Fi) specifically to steal an active session cookie.

Predictable Session Token

Guesses a valid session ID because the application generates them in a predictable pattern.

How It Works

Stage	Action	Example
1. Session Establishment	The user logs into a legitimate website, and the server issues a unique session ID (often stored in a cookie).	You log into your banking portal; the server gives your browser a session cookie: SID=abc123xyz.
2. Interception	The attacker intercepts the session ID while it is in transit or steals it from the user's local storage.	The attacker uses a sniffer on an open Wi-Fi network to capture the cookie SID=abc123xyz as you send it to the bank.
3. Impersonation	The attacker injects the stolen session ID into their own browser/tool.	The attacker opens their browser, uses a plugin to set their cookie to SID=abc123xyz, and navigates to the bank's site.
4. Exploitation	The server validates the stolen ID, granting the attacker full access to the user's active account.	The bank's server sees the valid cookie and logs the attacker directly into <i>your</i> account without needing a password.



Example Scenario: Session Hijacking on a Public Wi-Fi Network

Network Details

Device	IP Address
Wi-Fi Router	192.168.10.1
Victim Laptop	192.168.10.25
Attacker Laptop	192.168.10.30
Banking Website	https://securebank.com
Session Cookie	SID=9fA72Xc81LmP

1. Session Establishment

Action:

The user logs into the banking website successfully. After verifying the username and password, the server creates a unique session ID and stores it in the user's browser as a cookie.

Example:

1. Alice connects to **Free_Guest_WiFi**.
2. She opens <https://securebank.com>.
3. She enters her username and password.
4. The bank verifies the credentials and creates:

Session ID: SID=9fA72Xc81LmP

5. Alice's browser automatically includes this session cookie with every subsequent request, allowing her to remain logged in without re-entering her password.

2. Interception

Action:

The attacker obtains the victim's session cookie while it is exposed due to an insecure application, compromised network, malware, or another vulnerability.

Example:

Alice continues browsing her account.

The attacker captures the following session cookie:

Cookie:

SID=9fA72Xc81LmP

The attacker **does not know Alice's password**—only the active session identifier.

3. Impersonation

Action:

The attacker places the stolen session ID into their own browser so that requests appear to come from the authenticated user.

Example:

The attacker configures their browser with:

Cookie:

SID=9fA72Xc81LmP

Then they visit:

<https://securebank.com/account>

The request now carries the same session cookie as Alice's browser.

4. Exploitation

Action:

The server checks the session ID. Since it is still valid, it assumes the request belongs to the legitimate user and grants access.

Example:

The server finds:

Received Cookie:
SID=9fA72Xc81LmP

Stored Active Session:
SID=9fA72Xc81LmP

Because the values match, the server loads Alice's authenticated session.

The attacker can now view account information and perform any actions that the legitimate logged-in session is authorized to perform until the session expires or is invalidated.

Complete Workflow

1. Alice logs into SecureBank
2. Server creates Session ID
SID=9fA72Xc81LmP
3. Attacker obtains the Session ID
4. Attacker inserts the same Session ID into their browser
5. Server accepts the valid Session ID
6. Attacker gains access to Alice's active authenticated session

Real-World Example

REAL-WORLD EXAMPLE

An attacker on the same public Wi-Fi network as a victim sniffs an unencrypted session cookie from the victim's logged-in webmail session, then replays that cookie in their own browser to access the victim's inbox without ever needing the victim's password.

Prevention Measures

- Use HTTPS everywhere so session tokens can't be sniffed off the wire in plaintext.
- Regenerate session IDs after login to defeat session fixation.
- Use long, random, unpredictable session tokens.
- Set short session timeouts and re-authenticate for sensitive actions.
- Bind sessions to additional signals (IP/device fingerprint) so a stolen token alone isn't enough.
- Sanitize all user input to prevent XSS-based cookie theft.

Physical Security

Network protection isn't purely a software problem. No matter how strong your firewall is, you also need to guard against attacks from inside the building — statistically, insider threats are more likely than external ones.

Physical Access Defeats Everything Else

BEHIND LOCKED DOORS

If an intruder gets physical access to a server, none of your firewall, IDS, encryption, or PKI matters — physical access basically defeats all of it. Just like bomb-making information is findable online, lockpicking techniques are too. Claimed skill benchmarks: an average person can learn to pick a typical home lock in under 5 minutes with a few days' practice; dedicated attackers can pick a deadbolt in under 30 seconds. Night shifts, with fewer staff around, make unauthorized access easier for insiders. High-security organizations like the DoD sometimes spend \$5,000–\$10,000 on a single lock given life-or-death stakes.

Lock Types & Access Control

Rotary/Combination Locks

Harder to crack than standard deadbolts, but log nothing about who entered.

Card Access Systems

A scanned card logs entry time and can grant access to multiple doors without issuing physical keys.

Biometrics

Fingerprint or retinal scans can complement a lock as a second factor, or replace it entirely.

Biometric Limits

Not foolproof — one smartphone's facial recognition was fooled with just a photograph; pair biometrics with a second factor for real MFA.

Real Incidents Worth Remembering

- At a Hawaii military base, someone drove a pickup truck up, walked into an empty office, unplugged a computer, and carried it out — never caught, because staff assumed he was IT.
- Lesson: physical security is only as strong as its weakest link — every employee needs to stay alert, not just security staff.
- In many states/countries, merely possessing lockpicking tools (or certain hacking tools) can itself be a crime — know local laws before acquiring any.

Quick Revision

A one-glance summary of every attack category covered in this unit — use this as a final check before moving on.

Quick Revision

Password Hacking	Discovering or bypassing login credentials.
Keyloggers	Capturing keystrokes to steal typed credentials.
Rootkits	Stealth software hiding an attacker's continued presence.
Trojans	Malware disguised as legitimate software.
Backdoors	A hidden way to bypass normal authentication.
Malware (general)	Umbrella term for viruses, worms, Trojans and more.
Viruses	Malicious code that attaches to a host file to spread.
Worms	Self-replicating malware that spreads without a host file.

Quick Revision (cont.)

Spyware / Adware	Covertly monitors activity or tracks habits for ads.
Sniffers	Capture and inspect passing network traffic.
Eavesdropping / MitM	Passive listening vs. active interception of traffic.
DoS / DDoS	Floods a target to block legitimate access.
Buffer Overflow	Exploits unenforced memory limits to run attacker code.
Session Hijacking	Taking over an already-authenticated session.
Physical Security	Guarding devices and facilities from in-person access.